SDK API

Table of Contents

1. Introduction
    1.1. Terminology
2. Input Events
    2.1. Media Events
        2.1.1. Raw Video
        2.1.2. Raw Audio
    2.2. Analytics Events
        2.2.1. Motion Detection
        2.2.2. Tampering Detection
        2.2.3. Audio Detection
        2.2.4. Sound Classification
        2.2.5. Defocus Detection
        2.2.6. Alarm In
        2.2.7. Alarm Out
        2.2.8. ONVIF Metadata
    2.3. SD Card Events
        2.3.1. SD Card Inserted
        2.3.2. SD Card Removed
        2.3.3. SD Card Recording Started
        2.3.4. SD Card Recording Stopped
    2.4. Setting Changes Notification Events
        2.4.1. Network Interface
        2.4.2. Network Port
        2.4.3. Video Profile
        2.4.4. Audio Config
        2.4.5. Image Config
        2.4.6. Video Rotation Config
        2.4.7. Wisestream Config
        2.4.8. Exposure AI Config
        2.4.9. Storage Config
        2.4.10. Language Config
        2.4.11. Time Config
        2.4.12. Motion Detection Config
        2.4.13. Tampering Detection Config
        2.4.14. Audio Detection Config
        2.4.15. Sound Classification Config
        2.4.16. Defocus Detection Config
    2.5. Stop Application Events
        2.5.1. App Warning
        2.5.2. Stop Application
    2.6. Uploaded File Name Events

- 2.6.1. Uploaded File Name
3. Output Events
  - 3.1. Alarm Event
    - 3.1.1. Alarm Output ON
    - 3.1.2. Alarm Output OFF
  - 3.2. Misc Events
    - 3.2.1. FTP File Upload
    - 3.2.2. Send Email Notification
    - 3.2.3. Snapshot JPEG
    - 3.2.4. Set OSD Message
    - 3.2.5. Send Metadata
      - 3.2.5.1. Event Metadata
      - 3.2.5.2. Frame Metadata
    - 3.2.6. Write Event Log
    - 3.2.7. Set Dynamic Schema
      - 3.2.7.1. Metadata Schema
      - 3.2.7.2. EventStatus Schema
    - 3.2.8. MetaFrame Scheme
    - 3.2.9. MetaFrame Capability
    - 3.2.10. Event Status Check
    - 3.2.11. Event Status Changed
    - 3.2.12. Send Dynamic Metadat
  - 3.3. SD Card Events
    - 3.3.1. Get SD Card Status
    - 3.3.2. Start SD Card
    - 3.3.3. Stop SD Card
    - 3.3.4. Format SD Card
    - 3.3.5. Set Exclusive Mode
    - 3.3.6. Get Exclusive Mode Status
4. App Support Package
  - 4.1. Network Manager
    - 4.1.1. Sendable Events
    - 4.1.2. Receivable Events
    - 4.1.3. Sendable Events
      - 4.1.3.1. Start Service
      - 4.1.3.2. Stop Service
      - 4.1.3.3. Send Data
      - 4.1.3.4. Close Client
    - 4.1.4. Receivable Events
      - 4.1.4.1. New Client Connected
      - 4.1.4.2. Client Disconnected
      - 4.1.4.3. Client Data
      - 4.1.4.4. Server Data
  - 4.2. SUNAPI Requester
    - 4.2.1. Sendable Events
      - 4.2.1.1. Request SUNAPI

4.2.2. Receivable Events
4.2.2.1. SUNAPI Response
4.2.2.2. Ready for Use SUNAPI
5. Event API Reference Table
5.1. Input Event API Reference Table
5.2. Output Event API Reference Table
5.3. App Support Package API Reference Table

1. Introduction

Hanwha Vision Open Platform SDK provides the following supports to develop applications.

- Input Events
- Output Events
- App Support Package
- Event API Reference Table

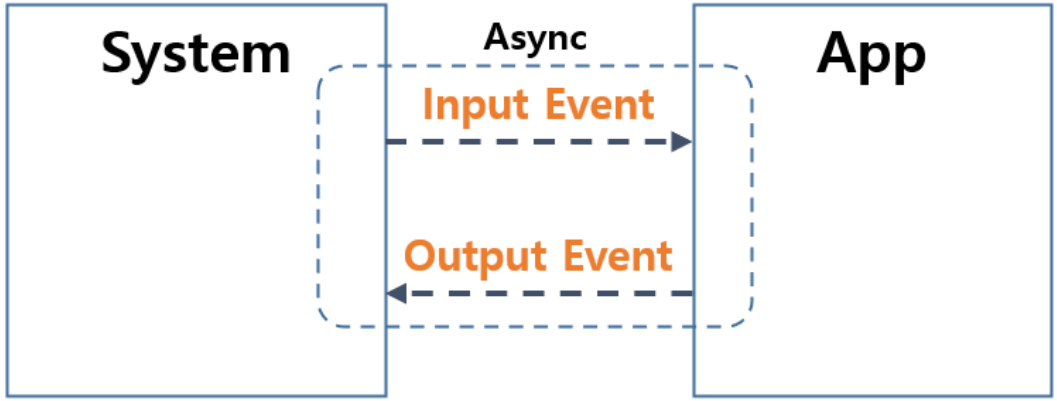
This document contains the following information

- Supported input and output event details
- Information about the packages needed for app development.

1.1. Terminology

Events are processed asynchronously and are named Input/Output Events based on their direction. An Input Event is an event that comes from outside into the app, while an Output Event is an event that goes from the app to the outside.

- Input Events
- Output Events



> *Note This document is an API reference. For guidelines related to OpenPlatform development, please refer to the* `HanwhaVision OpenPlatform Programming Guide` *document.*

2. Input Events

2.1. Media Events

The media events allow you to receive the captured audio or video data. Open platform supports the following events:

- **SourceNames** Registration

Event type	Source (#:max channel)	Group name
Raw Video	SPMgrVideoRaw_0 ~ SPMgrVideoRaw_#	GroupSPMgrVideoRaw2
Raw Audio	SPMgrAudioRaw	SRMgrAudioRawGroup2

- **Header** and **Event Type**

Event type	Header file	Enum type
Raw Video	i_p_stream_provider_manager_video_raw.h	IPStreamProviderManagerVideoRaw::EEventType::eVideoRawData
Raw Audio	i_p_stream_provider_manager_audio_raw.h	IPStreamProviderManagerAudioRaw::EEventType::eAudioRawData

2.1.1. Raw Video

With this event, you will receive raw video data by registering Source and Group. The received RawVideo Event contains the following information about the RawImage:

- Parameters
  - IPVideoFrameRaw

Variable	Type	Description
raw_img_	RawImage	raw image frame

  - RawImage

Variable	Type	Description
chan_id	uint32_t	channel id which is video frame captured
seq	uint32_t	sequence number of the video frame
pts	uint64_t	presentation time of the video frame
format	raw_format_type	image format of the video frame
width	uint32_t	width of the video frame
height	uint32_t	height of the video frame
pitch	uint32_t	pitch of the video frame
num_planes	uint32_t	total number of planes

Variable	Type	Description
plane	plane struct	plane type and memory info for each plane
next	RawImage*	pointer to the next item in linked list
type	raw_image_type_t	raw image address mapping type
dmabuf_fd	int32_t	file descriptor for dmabuf

- raw_format_type

```cpp
enum raw_format_type {
  RAW_FMT_UNKNOWN,
  RAW_FMT_YUV422,
  RAW_FMT_YUV420,
  RAW_FMT_NV12,
  RAW_FMT_NV21,
  RAW_FMT_RGBA,
  RAW_FMT_BGRA,
  RAW_FMT_RGB888,
  RAW_FMT_BGR888,
  RAW_FMT_RGB565,
  RAW_FMT_BGR565,
  RAW_FMT_ONLY_Y,
  RAW_FMT_UYVY,
  RAW_FMT_YVU420,
  RAW_FMT_YVU422,
  RAW_FMT_YUYV,
  RAW_FMT_YVYU,
  RAW_FMT_ARGB8888,
  RAW_FMT_ARGB1555,
  RAW_FMT_ARGB4444,
};
```

- plane

Variable	Type	Description
type	raw_plane_type	plane type
phy_addr	uint64_t	physical address of the plane
vir_addr	vir_addr	user-space address of the plane
addr_offset	unsigned long	plane's physical address offset from the memory-mapped physical address start address

  - raw_plane_type

```cpp
enum raw_plane_type {
  RAW_PLANE_UNKNOWN,
  RAW_PLANE_Y,
  RAW_PLANE_U,
  RAW_PLANE_V,
  RAW_PLANE_UV,
  RAW_PLANE_YUV,
};
```

- Example
  - Header file

```cpp
bool Classification::ProcessAEvent(Event *event)
{
  switch (event->GetType())
  {
      case static_cast<int32_t>(IPStreamProviderManagerVideoRaw::EEventType::eVideoRawData):
          ProcessRawVideo(event);
          break;
  }
  return result;
}
```

  - Source file

```cpp
void Classification::ProcessRawVideo(Event* event) {
  ...
  IPVideoFrameRaw* raw_frame = new ("GetImage") IPLVideoFrameRaw();
  raw_frame->DeserializeBaseObject(raw_frame, ret);

  std::shared_ptr<RawImage> img(raw_frame->GetRawImage());
  ...
  Inference(img);
  ...
}

void Classification::Inference(shared_ptr<RawImage> img)
{
  ...
  for (auto *image = img.get(); image; image = image->next)
  {
    if (image->width < 3840 && image->height < 2160)
    {
      ...
    }
  }
}
```

2.1.2. Raw Audio

With this event, you will receive raw audio data provided in the PCM (Pulse Code Modulation) format. Along with the data, you will know the following information:

```
- Data Size: The total size of the audio data
- Sampling Rate: The rate at which the audio data is sampled. The number of samples per unit of time.
- Bit Depth: The number of bits per sample in the audio data, with higher bit depths typically indicating higher audio
quality.
- Time: The exact time at which the audio event occurred.
- Amplitude: The intensity or magnitude of the audio signal.
- Codec Type: The type of codec, which is in PCM (Pulse Code Modulation) format.
- Channel Number
- Track Number: The specific track or stream to which the audio data belongs.
- Device Presentation Time Stamp: A timestamp indicating the moment when the audio data is played or recorded, based on
the device's time.
```

- Parameters
  - IPAudioFrameRaw

Variable	Type	Description
raw_audio	PCM_FRAME	audio frame

  - PCM_FRAME

Variable	Type	Description
Audio header	HDR	GetHeader()
Data	unsigned char*	Data()
Data size	uint32_t	DataSize()

  - HDR

Variable	Type	Description
channel		Unused
track		Unused
codectype	uint16_t	CODEC_TYPE, only PCM is supported
packet_time_sec	uint32_t	sec
packet_time_usec	uint32_t	usec
device_pts		Unused
samplerate	unsigned int	audio sampling rate
amplitude	unsigned char	audio bit

  - Codec Type

Type
PCM

- Example

```
bool SampleComponent::ProcessAEvent(Event* event) {
switch (event->GetType()) {
 case static_cast<int>(IPStreamProviderManagerAudioRaw::EEventType::eAudioRawData): {
   auto blob = event->GetBlobArgument();
   std::pair<std::variant<BaseObject*, char*>, uint64_t> ret((char*)blob.GetRawData(), blob.GetSize());

   IPAudioFrameRaw audio_frame_raw;
   audio_frame_raw.DeserializeBaseObject(&audio_frame_raw, ret);

   auto raw_audio = audio_frame_raw.GetRawAudio();
   if (raw_audio == nullptr) {
     std::cout << "ERROR! raw audio is nullptr" << std::endl;
     return;
   }

   if (raw_audio->IsPCM()) {
     printf("codec type is PCM\n");
   }
   printf("codectype : %u\n", raw_audio->GetHeader()->codectype);
   uint64_t timestamp = raw_audio->GetHeader()->packet_time_sec * (uint64_t)1000 + raw_audio->GetHeader()-
>packet_time_usec / 1000;
   printf("timestamp : %lu\n", timestamp);
   printf("samplerate : %u\n", raw_audio->GetHeader()->samplerate);
   printf("amplitude : %d\n", raw_audio->GetHeader()->amplitude);

   auto data = raw_audio->Data();
   auto data_size = raw_audio->DataSize();
   printf("data size : %d\n", data_size);

   break;
 ...
}
...
```

2.2. Analytics Events

The analytics events allow you to receive the results of the video and audio analysis features provided by default in the camera as events. To receive this event, enable analysis features on the camera web page.

- **SourceNames** Registration

Event type	Source	Group name
Motion detection	MotionDetector_0 ~ MotionDetector_#	OpenSDK::Analytics
Tampering detection	TamperingDetector_0 ~ amperingDetector_#	OpenSDK::Analytics
Audio detection	AudioDetector	OpenSDK::Analytics
Sound classification	SoundClassification	OpenSDK::Analytics

Event type	Source	Group name
Defocus detection	DefocusDetector_0 ~ DefocusDetector_#	OpenSDK::Analytics
Alarm in	ConfigurableAlarmIn	OpenSDK::AlarmIO
Alarm out	ConfigurableAlarmOut	OpenSDK::AlarmIO
ONVIF metadata	MetadataManager_0 ~ MetadataManager_#	Metadata::OpenApp

- Header file and Enum event type

Event type	Header file	Enum type
Motion detection	i_analytics_detector.h	IAnalyticsDetector::EEventType::eNotifyEventOccured
Tampering detection	i_analytics_detector.h	IAnalyticsDetector::EEventType::eNotifyEventOccured
Audio detection	i_analytics_detector.h	IAnalyticsDetector::EEventType::eNotifyEventOccured
Sound classification	i_analytics_detector.h	IAnalyticsDetector::EEventType::eNotifyEventOccured
Defocus detection	i_analytics_detector.h	IAnalyticsDetector::EEventType::eNotifyEventOccured
Alarm in	i_configurable_alarm_in.h	IConfigurableAlarmIn::EEvent::eNotifyEventOccured
Alarm out	i_configurable_alarm_out.h	IConfigurableAlarmOut::EEvent::eNotifyEventOccured
ONVIF metadata	i_p_metadata_manager.h	IPMetadataManager::EEventType::eMetadataRequest

2.2.1. Motion Detection

With this event, you will be notified when motion is detected within the area you set. You will also know the occurrence time and detection status. To receive motion-detection events, go to [Setting]>[Analysis]>[Motion Detection]. If you want to install the application on a multi-channel camera, you can simply associate each channel to receive detection events.

- Parameters
  You can receive detecting-event information in JSON format. See the explanation of each key and example results below.
  - SerializableString

JSON key name	Type	Description
EventType	string	Detected event type: MotionDetection
MessageTime	string	Detected event occurrence time
State	boolean	Detected event status

```JSON
{
  "EventType": "MotionDetection",
  "MessageTime": "2023-06-22T06:15:24.410Z",
  "State": true
}
```

- Example: Receives motion-detection events and prints contents

```CPP
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IAnalyticsDetector::EEventType::eNotifyEventOccured): {
      auto ret = eventToArgumentBuffer(event);
      SerializableString s;
      s.DeserializeBaseObject(&s, ret);
      std::cout << s.GetString() << std::endl;
      break;
    }
    ...
  }
  ...
}
```

2.2.2. Tampering Detection

With this event, you will be notified when the screen is blocked, or the camera position changes. You will also get the occurrence time and detection status. To receive tampering-detection events, go to [Settings]>[Analysis]>[Tampering detection]. If you want to install the application on a multi-channel camera, you can simply associate each channel to receive detection events.

- Parameters
  You can get detecting-event information in JSON format. See the explanation of each key and example results below.
  - SerializableString

JSON key name	Type	Description
EventType	string	Detected event type: TamperingDetection
MessageTime	string	Detected event occurrence time
State	boolean	Detected event status

```JSON
{
  "EventType": "TamperingDetection",
  "MessageTime": "2023-06-22T06:15:24.410Z",
  "State": true
}
```

- Example: Receives tampering-detection events and prints contents.

  The example is identical to the one for Motion Detection. To see the example, refer to Motion Detection.

2.2.3. Audio Detection

With this event, you will be notified when audio above a set level is detected. You will also get the occurrence time and detection status. To receive audio-detection events, go to [Settings]>[Analysis]>[Audio detection].

- Parameters
  You can get detecting-event information in JSON format. See the explanation of each key and example results below.
  - SerializableString

JSON key name	Type	Description
EventType	string	Detected event type: AudioDetection
MessageTime	string	Detected event occurrence time
State	boolean	Detected event status

```JSON
{
  "EventType": "AudioDetection",
  "MessageTime": "2023-06-22T06:15:24.410Z",
  "State": true
}
```

- Example: Receives audio-detection events and prints contents.

  The example is identical to the one for Motion Detection. To see the example, refer to Motion Detection.

2.2.4. Sound Classification

With this event, you will know the results of classifying the type of sound detected during video recording. You will also get the occurrence time and detection status. To receive sound-classification events, go to [Settings]>[Analysis]>[Sound classification].

- Parameters
  You can get detecting-event information in JSON format. See the explanation of each key and example results below.
  - SerializableString

JSON key name	Type	Description
EventType	string	Detected event type: SoundClassification
MessageTime	string	Detected event occurrence time
Classification	string	Detected event result: Scream, Gunshot, Explosion, Crashing glass
State	boolean	Detected event status

```JSON
{
  "EventType": "SoundClassification",
  "MessageTime": "2023-06-22T06:15:24.410Z",
  "Classification": "Scream",
  "State": true
}
```

- Example: Receives sound-classification events and prints contents.

  The example is identical to the one for Motion Detection. To see the example, refer to Motion Detection.

2.2.5. Defocus Detection

With this event, you will be notified when the camera lens focus is detected to be blurred. You will also get the occurrence time and detection status. To receive defocus-detection event, go to [Settings] [Analysis]>[Defocus detection]. If you want to install the application on a multi-channel camera, you can simply associate each channel to receive detection events.

- Parameters
  You can get detecting event information in JSON format. See the explanation of each key and example results below:
  - SerializableString

JSON key name	Type	Description
EventType	string	Detected event type: Defocus Detection
MessageTime	string	Detected event occurrence time
State	boolean	Detected event status

```JSON
{
  "EventType": "DefocusDetection",
  "MessageTime": "2023-06-22T06:15:24.410Z",
  "State": true
}
```

- Example: Receives defocus-detection events and prints contents.

  The example is identical to the one for Motion Detection. To see the example, refer to Motion Detection.

2.2.6. Alarm In

To receive alarm-input events, go to [Settings]>[Analysis]>[Alarm IO].

- Parameters
  You can get detecting event information in JSON format. See the explanation of each key and example results below:
  - SerializableString

JSON key name	Type	Description
EventName	string	Detected event type: AlarmInput
Time	string	Detected event occurrence time
State	boolean	Detected event state
Channel	integer	Channel number
SourceID	integer array	Logical port index

```JSON
{
  "AlarmInput": {
    "Channel": 0,
    "State": true,
    "Time": "2024-05-07T08:27:52.415+00:00",
    "EventName": "AlarmInput",
    "Source": {
      "SourceID": [
        1
      ]
    },
    "Data": {
      "State": [
        true
      ]
    }
  }
}
```

- Example: Receives alarm-in events and prints contents.

```CPP
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IConfigurableAlarmIn::EEvent::eNotifyEventOccured): {
      auto ret = eventToArgumentBuffer(event);
      SerializableString s;
      s.DeserializeBaseObject(&s, ret);
      std::cout << s.GetString() << std::endl;
      break;
    }
    ...
  }
  ...
}
```

2.2.7. Alarm Out

To receive alarm-output events, go to [Settings]>[Analysis]>[Alarm IO].

- Parameters
  You can get detecting event information in JSON format. See the explanation of each key and example results below.
  - SerializableString

JSON key name	Type	Description
EventName	string	Detected event type: AlarmOutput
Time	string	Detected event occurrence time
State	boolean	Detected event state
Channel	integer	Channel number
SourceID	integer array	Logical port index

```JSON
{
  "AlarmOutput": {
    "Channel": 0,
    "State": true,
    "Time": "2024-05-07T08:38:11.511+00:00",
    "EventName": "AlarmOutput",
    "Source": {
      "SourceID": [
        1
      ]
    },
    "Data": {
      "State": [
        true
      ]
    }
  }
}
```

- Example: Receives alarm-out events and prints contents.

```CPP
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IConfigurableAlarmOut::EEvent::eNotifyEventOccured): {
      auto ret = eventToArgumentBuffer(event);
      SerializableString s;
      s.DeserializeBaseObject(&s, ret);
      std::cout << s.GetString() << std::endl;
      break;
    }
    ...
  }
  ...
}
```

2.2.8. ONVIF Metadata

ONVIF Metadata performs roles, such as device management, event notification, timestamp, and media stream control based on the Real-Time Streaming Protocol (RTSP). You will receive ONVIF metadata information sent from the camera. For more information about ONVIF metadata, follow the link: (https://www.onvif.org/specs/srv/analytics/ONVIF-VideoAnalytics-Service-Spec-v221.pdf).

- Parameters
  - MetadataOutput

Variable	Type	Description
channel	int	Channel info
output	string	Metadata

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPMetadataManager::EEventType::eMetadataRequest): {
      auto ret = eventToArgumentBuffer(event);
      IPMetadataManager::MetadataOutput param;
      param.DeserializeBaseObject(&param, ret);
      std::cout << param.output().c_str() << std::endl;
      std::cout << param.channel() << std::endl;
      break;
    }
    ...
  }
}
```

2.3. SD Card Events

SD-card events allow you to identify when the SD card is inserted or that recording is not in progress. To receive SD-card events, allow the application to access the SD card. Refer to **Hanwha_Vision_Open_Platform_Programming_Guide** for your application's 'app_manifest.json'.

- **SourceNames** Registration

Event type	Source	Group name
SD Card Inserted	DeviceIoManager	DeviceIoManager::Sdcard
SD Card Removed	DeviceIoManager	DeviceIoManager::Sdcard
SD Card Recording Started	OpenPlatform	AppMessage
SD Card Recording Stopped	OpenPlatform	AppMessage

- **Header** and **Event type**

Event type	Header file	Event type (enum)
SD Card Inserted	i_device_io_manager.h	IDeviceIoManager::EEventType::eSdcardInserted
SD Card Removed	i_device_io_manager.h	IDeviceIoManager::EEventType::eSdcardRemoved
SD Card Recording Started	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eSDCardStarted
SD Card Recording Stopped	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eSDCardStopped

2.3.1. SD Card Inserted

With this event, you will be notified when an SD card is inserted into the slot.

- Parameters
  - SerializableString

Variable	Type	Description
sd_index	int	SD card slot number
block_index	int	SD card block number

```JSON
{"sd_index":1,"block_index":0}
```

- Example

```CPP
bool SampleComponent::ProcessAEvent(Event* event) {
switch (event->GetType()) {
  case static_cast<int32_t>(IDeviceIoManager::EEventType::eSdcardInserted): {
    auto ret = eventToArgumentBuffer(event);
    SerializableString param;
    param.DeserializeBaseObject(&param, ret);

    rapidjson::Document doc;
    doc.Parse(param.GetString());

    printf("sd_index", doc["sd_index"].GetInt());
    printf("block_index", doc["block_index"].GetInt());
    break;
  }
  ...
}
```

2.3.2. SD Card Removed

With this event, you will be notified when an SD card is removed from a slot.

- Parameters
  - SerializableString

Variable	Type	Description
sd_index	int	SD card slot number
block_index	int	SD card block number

```JSON
{"sd_index":1,"block_index":0}
```

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
switch (event->GetType()) {
  case static_cast<int32_t>(IDeviceIoManager::EEventType::eSdcardRemoved): {
    auto ret = eventToArgumentBuffer(event);
    SerializableString param;
    param.DeserializeBaseObject(&param, ret);

    rapidjson::Document doc;
    doc.Parse(param.GetString());

    printf("sd_index", doc["sd_index"].GetInt());
    printf("block_index", doc["block_index"].GetInt());
    break;
  }
  ...
}
```

2.3.3. SD Card Recording Started

With this event, you will be notified when SD card recording starts.

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
switch (event->GetType()) {
  case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eSDCardStarted): {
    ...
    break;
  }
  ...
}
```

2.3.4. SD Card Recording Stopped

With this event, you will be notified when SD card recording stops.

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
switch (event->GetType()) {
  case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eSDCardStopped): {
    ...
    break;
  }
  ...
}
```

2.4. Setting Changes Notification Events

Setting-changes-notification events allow you to identify how many times the channel settings were changed. The supported setting-change events are as follows:

- **SourceNames** Registration

Event type	Source	Group name
Network Interface	OpenPlatform	SettingChange
Network Port	OpenPlatform	AppMessage
Video Profile	OpenPlatform	SettingChange
Audio Config	OpenPlatform	SettingChange
Image Config	OpenPlatform	SettingChange
Video Rotation Config	OpenPlatform	AppMessage
Wisestream Config	OpenPlatform	SettingChange
Exposure AI Config	OpenPlatform	SettingChange
Storage Config	OpenPlatform	SettingChange
Language Config	OpenPlatform	AppMessage
Time Config	OpenPlatform	AppMessage
Motion Detection Config	OpenPlatform	SettingChange
Tampering Detection Config	OpenPlatform	SettingChange
Audio Detection Config	OpenPlatform	SettingChange
Sound Classification Config	OpenPlatform	SettingChange
Defocus Detection Config	OpenPlatform	SettingChange

- **Header** and **Event Type**

Event type	Header file	Event type (enum)
Network Interface	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eNetworkSettingChanged
Network Port	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eUpdateAuthInfo
Video Profile	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eProfileSettingChanged
Audio Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAudioSettingChanged
Image Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eImageSettingChanged
Video Rotation Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eUpdateVideoRotation

Event type	Header file	Event type (enum)
Wisestream Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eWiseStreamSettingChanged
Exposure AI Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eExposureAISettingChanged
Storage Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eRecordSettingChanged
Language Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eUpdateSystemSetting
Time Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eTimeSettingChanged
Motion Detection Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged
Tampering Detection Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged
Audio Detection Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged
Sound Classification Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged
Defocus Detection Config	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged

2.4.1. Network Interface

With this event, you will be notified when network settings change. This event occurs when you change network-related settings such as DNS, Gateway, and IP on the web. To receive network-interface events, go to [Settings]>[Basic]>[IP & Port] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eNetworkSettingChanged): {
      printf("Network Interface setting is changed!");
      break;
    }
  }
  ...
}
```

2.4.2. Network Port

With this event, you will be notified when the HTTP and HTTPS Port number settings are changed. To receive network-port events, go to [Settings]>[Basic]>[IP & Port]>[Port] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eUpdateAuthInfo): {
      printf("Network port number is changed!");
      break;
    }
  }
  ...
}
```

2.4.3. Video Profile

With this event, you will be notified when video profile settings change. To receive video-profile events, go to [Settings]>[Camera]>[Profile setup] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
channel	int	Channel number whose settings have been changed

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eProfileSettingChanged): {
      printf("Profile setting is changed! Channel is %d", event->GetChannel());
      break;
    }
  }
  ...
}
```

2.4.4. Audio Config

With this event, you will be notified when audio settings change. To receive audio-config events, go to [Settings]>[Video & Audio]>[Audio setup] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eAudioSettingChanged): {
      printf("Audio setting is changed!");
      break;
    }
  }
  ...
}
```

2.4.5. Image Config

With this event, you will be notified when image settings change. To receive image-config events, go to [Settings]>[Video & Audio]>[Camera setup] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
channel	int	Channel number whose settings have been changed

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eImageSettingChanged): {
      printf("Image setting is changed! Channel is %d", event->GetChannel());
      break;
    }
  }
  ...
}
```

2.4.6. Video Rotation Config

With this event, you will be notified when the video rotation settings change. To receive video-rotation-config events, go to [Settings]>[Video & Audio]>[Video setup]>[Video rotation] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
channel	int	Channel number whose settings have been changed

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eUpdateVideoRotation): {
      printf("Video rotation setting is changed! Channel is %d", event->GetChannel());
      break;
    }
  }
  ...
}
```

2.4.7. Wisestream Config

With this event, you will be notified when WiseStream settings change. To receive Wisestream-config events, go to [Settings]>[Video & Audio]>[Wise Stream] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
channel	int	Channel number whose settings have been changed

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eWiseStreamSettingChanged): {
      printf("WiseStream setting is changed! Channel is %d", event->GetChannel());
      break;
    }
  }
  ...
}
```

2.4.8. Exposure AI Config

With this event, you will be notified when Exposure AI settings change. This event occurs when you change the AI-based prefer shutter control checkbox and the Wise NR II setting in SSNR in [Settings]>[Camera Setup]>[Exposure] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
channel	int	Channel number whose settings have been changed

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eExposureAISettingChanged): {
      printf("Exposure AI setting is changed! Channel is %d", event->GetChannel());
      break;
    }
  }
  ...
}
```

2.4.9. Storage Config

With this event, you will be notified when storage settings change. To receive storage-config events, go to [Settings]>[Event]>[Storage] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eRecordSettingChanged): {
      printf("Storage setting is changed!);
      break;
    }
  }
  ...
}
```

2.4.10. Language Config

With this event, you will be notified when system settings change. System settings include device name, location, language, etc. To receive language-config events, go to [Settings]>[System]>[Product information] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eUpdateSystemSetting): {
      printf("System setting is changed!);
      break;
    }
  }
  ...
}
```

2.4.11. Time Config

With this event, you will be notified when time settings change. To receive time-config events, go to [Settings]>[Basic]>[Date & Time] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters
  None.

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eUpdateTime): {
      printf("Time setting is changed!);
      break;
    }
  }
  ...
}
```

2.4.12. Motion Detection Config

With this event, you will be notified when motion-detection settings change. To receive motion-detection-config events, go to [Settings]>[Analytics]>[Motion detection] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
detector name	string	Detector name

```
MotionDetection
```

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged): {
      auto ret = eventToArgumentBuffer(event);
      SerializableString s;
      s.DeserializeBaseObject(&s, ret);
      std::cout << s.GetString() << std::endl;
      break;
    }
  }
  ...
}
```

2.4.13. Tampering Detection Config

With this event, you will be notified when tampering-detection settings change. To receive tampering-detection-config events, go to [Settings]>[Analytics]>[Tampering detection] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
detector name	string	Detector name

```
TamperingDetection
```

- Example
  The example is identical to the one for Motion Detection Config. To see the example, refer to Motion Detection Config.

2.4.14. Audio Detection Config

With this event, you will be notified when audio-detection settings change. To receive audio-detection-config events, go to [Settings]>[Analytics]>[Audio detection] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
detector name	string	Detector name

```
AudioDetection
```

- Example
  The example is identical to the one for Motion Detection Config. To see the example. refer to Motion Detection Config.

2.4.15. Sound Classification Config

With this event, you will be notified when the sound-classification-detection settings change. To receive sound-classification-config events, go to [Settings]>[Analytics]>[Sound Classification] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
detector name	string	Detector name

```
SoundClassification
```

- Example
  The example is identical to the one for Motion Detection Config. To see the example, refer to Motion Detection Config.

2.4.16. Defocus Detection Config

With this event, you will be notified when the defocus-detection settings change. To receive defocus-detection-config events, go to [Settings]>[Analytics]>[Defocus detection] on the web page. (Depending on models, the setting location may not exist or may vary.)

- Parameters

Variable	Type	Description
detector name	string	Detector name

```
DefocusDetection
```

- Example
  The example is identical to the one for Motion Detection Config. To see the example, refer to Motion Detection Config.

2.5. Stop Application Events

Stop-application events allow you to identify that a critical event has occurred, such as high memory or CPU usage, requiring the application to stop.

- **SourceNames** Registration

Event type	Source	Group name
App Warning	OpenPlatform	AppMessage
Stop Application	OpenPlatform	AppMessage
Error notification event	-	-

- Header file and Event type (enum)

Event type	Header file	Enum type
App Warning	i_p_open_platform_manager.h,open_platform_define.h	IPOpenPlatformManager::EAppEventType::eAppWarning
Stop Application	i_p_open_platform_manager.h,open_platform_define.h	IPOpenPlatformManager::EAppEventType::eAppStop
Error notification event	-	-

2.5.1. App Warning

With this event, you will be notified when the total resource (CPU, Memory) use surpasses the limit of 90%. The event message includes the application name and resource information where the problem occurred. If there are multiple applications, the camera sends the app-warning event to the application that is using the most resources and has the lowest priority.

- Parameters
  - SerializableString

Variable	Type	Description
AppName	string	Name of the application where the problem occurred
Message	string	Problem details

```JSON
{
  "app_name": "SampleApp_Name",
  "message": "CPULimitOver",
}
```

- Message List

```TEXT
- "OK",
- "CPULimitOver",
- "MemoryLimitOver",
- "CPUMemoryLimitOver",
```

- Example

```CPP
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case (int32_t)IPOpenPlatformManager::EAppEventType::eAppWarning: {
      auto ret = eventToArgumentBuffer(event);
      AppMsgParam appMsgParam("");
      appMsgParam.DeserializeBaseObject(&appMsgParam, ret);
      printf( "[CHECK] Recv Warning Event [app_name: %s] [msg: %s]\n",
        appMsgParam.GetAppName().c_str(), appMsgParam.GetMessage().c_str());
      break;
    }
    ...
  }
  ...
}
```

2.5.2. Stop Application

With this event, you will be notified when the app-warning event occurs 3 times. When the application receives this event, the application stops running. After termination, the application must send a Stop-done event to the camera.

- Parameters
  - SerializableString

Variable	Type	Description
AppName	string	Application names to be stopped

```JSON
{
  "app_name": "SampleApp_Name"
}
```

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case (int32_t)IPOpenPlatformManager::EAppEventType::eAppStop: {
      auto ret = eventToArgumentBuffer(event);
      AppMsgParam appMsgParam("");
      appMsgParam.DeserializeBaseObject(&appMsgParam, ret);
      printf("[CHECK] Recv StopApp Event [app_name: %s]\n", appMsgParam.GetAppName().c_str());
      break;
    }
    ...
  }
  ...
}
```

2.6. Uploaded File Name Events

When a new file is uploaded to the application, you can receive the application name and the name of the new file through this event.

- Uploading files
  For more information on how to upload files, contact us. We will provide you with the **SUNAPI** document that includes the information. When you upload an application, the file is uploaded to the ./app folder within the application.

```
$ curl -v --digest -u {ID}:{PASSWORD} -F DataFile=@{UPLOAD FILE NAME};type=application/octet-stream "http://{Camera IP}/stw-cgi/opensdk.cgi?msubmenu=apps&action=update&AppID={APP ID}" -H "Expect:"
```

- **SourceNames** Registration

Event type	Source	Group name
Uploaded File Name	OpenPlatform	AppMessage

- **Header** and **Event Type**

Event type	Header file	Enum type
Uploaded File Name	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eUploadFile

2.6.1. Uploaded File Name

With this event, you will be notified when a new file is uploaded inside the application. You will receive the name of the uploaded file.

- Parameters
  - AppMsgParam

Variable	Type	Description
app name	string	The name of the app where the file was uploaded
message	string	Uploaded file name

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eUploadFile): {
      if (event == nullptr) {
        printf("event is nullptr!");
        return false;
      }

      auto ret = eventToArgumentBuffer(event);
      AppMsgParam param;
      param.DeserializeBaseObject(&param, ret);
      auto app_name = param.GetAppName();
      auto message = param.GetMessage();

      printf("app name : %s", app_name.c_str());
      printf("upload file name : %s", message.c_str());
      break;
    }
    ...
  }
  ...
}
```

3. Output Events

3.1. Alarm Event

Alarm events allow you to turn the alarm out events on or off. The output events are as follows:

- **ReceiverNames** Registration

Event type	SymbolName	RealName
Alarm Output ON	-	ConfigurableAlarmOut
Alarm Output OFF	-	ConfigurableAlarmOut

- Header file and Event type (enum)

Event type	Header file	Enum type
Alarm Output ON	i_configurable_alarm_out.h	IConfigurableAlarmOut::EEvent::eRelayRequest
Alarm Output OFF	i_configurable_alarm_out.h	IConfigurableAlarmOut::EEvent::eRelayRequest

- Common Parameters
  - RelayInstance::Builder

Function	Parameter	Description
SetAction	string mode	Set on/off mode
SetChannel	uint32_t channel	Set channel

Function	Parameter	Description
SetDuration	uint32_t duration	Set duration
SetCSetStatehannel	string state	Set state

- IConfigurableAlarmOut::RelayRequest

Function	Parameter	Description
ParseRequest	Event* event	Parse the event request message and store it in relay_instance.
relay_instance	-	Returns the relay_instance.

- RelayInstance

Function	Parameter	Description
GetChannel	-	Returns the channel information.
GetDuration	-	Returns the duration information.
GetAction	-	Returns the action information.
GetState	-	Returns the state information.

- Replied Message
  - You can receive response messages for alarm On/off events using the **SendReplyEvent** function.The response message in the form of IConfigurableAlarmOut::RelayRequest is as follows:

```cpp
void SampleComponent::AlarmOutputOn(Event* event) {
  if (event->IsReply()) {
    IConfigurableAlarmOut::RelayRequest response;
    response.ParseRequest(event);

    RelayInstance relay_instance = response.relay_instance();
    std::stringstream log("");
    log << std::endl
        << "*************************************************" << std::endl
        << "I/O Port Channel: " << relay_instance.GetChannel() << std::endl
        << "Duration: " << relay_instance.GetDuration() << std::endl
        << "Action: " << relay_instance.GetAction() << std::endl
        << "State: " << relay_instance.GetState() << std::endl
        << "*************************************************" << std::endl;
    LOG(GetObjectName(), __FUNCTION__, __LINE__, log.str());
    return true;
  }
  ...
}
```

3.1.1. Alarm Output ON

This event enables the alarm output.

- Example

```cpp
void SampleComponent::AlarmOutputOn(Event* event) {
  if (event->IsReply()) {
    IConfigurableAlarmOut::RelayRequest response;
    response.ParseRequest(event);

    RelayInstance relay_instance = response.relay_instance();
    std::stringstream log("");
    log << std::endl
        << "*************************************************" << std::endl
        << "I/O Port Channel: " << relay_instance.GetChannel() << std::endl
        << "Duration: " << relay_instance.GetDuration() << std::endl
        << "Action: " << relay_instance.GetAction() << std::endl
        << "State: " << relay_instance.GetState() << std::endl
        << "*************************************************" << std::endl;
    LOG(GetObjectName(), __FUNCTION__, __LINE__, log.str());
    return true;
  }
  RelayInstance::Builder builder;

  std::string_view mode = "on";
  std::string_view channel = "1";

  builder.SetAction(mode.data()).SetChannel(atoi(channel.data())).SetDuration(1);

  SendReplyEvent("ConfigurableAlarmOut", static_cast<int32_t>(IConfigurableAlarmOut::EEvent::eRelayRequest), 0,
                 new ("RelayRequest") IConfigurableAlarmOut::RelayRequest(builder.Build()));
}
```

3.1.2. Alarm Output OFF

This event disables the alarm output.

- Example

```cpp
void SampleComponent::AlarmOutputOff(Event* event) {
  if (event->IsReply()) {
    IConfigurableAlarmOut::RelayRequest response;
    response.ParseRequest(event);

    RelayInstance relay_instance = response.relay_instance();
    std::stringstream log("");
    log << std::endl
        << "*************************************************" << std::endl
        << "I/O Port Channel: " << relay_instance.GetChannel() << std::endl
        << "Duration: " << relay_instance.GetDuration() << std::endl
        << "Action: " << relay_instance.GetAction() << std::endl
        << "State: " << relay_instance.GetState() << std::endl
        << "*************************************************" << std::endl;
    LOG(GetObjectName(), __FUNCTION__, __LINE__, log.str());
    return true;
  }
  RelayInstance::Builder builder;

  std::string_view mode = "off";
  std::string_view channel = "1";

  builder.SetAction(mode.data()).SetChannel(atoi(channel.data())).SetDuration(1);

  SendReplyEvent("ConfigurableAlarmOut", static_cast<int32_t>(IConfigurableAlarmOut::EEvent::eRelayRequest), 0,
                 new ("RelayRequest") IConfigurableAlarmOut::RelayRequest(builder.Build()));
}
```

3.2. Misc Events

- **SourceNames** Registration

Event type	Source	Group Name
Metadata Schema	OpenEventDispatcher	OpenSDK::Common::EventSource
Event Status Schema	OpenEventDispatcher	OpenSDK::Common::EventSource
Event Status Check	OpenEventDispatcher	OpenSDK::Common::EventSource

- **ReceiverNames** Registration

Event type	Symbol name	Real name
FTP File Upload	OpenPlatform	EComponents::eOpenPlatform
Send Email Notification	OpenPlatform	EComponents::eOpenPlatform
Snapshot JPEG	OpenPlatform	EComponents::eOpenPlatform
Set OSD message	SRMgrVideo	SRMgrVideo_0 ~ SRMgrVideo_#
Send Metadata(Event Metadata)	MetadataManager	MetadataManager_0 ~ MetadataManager_#
Send Metadata(Frame Metadata)	MetadataManager	MetadataManager_0 ~ MetadataManager_#
Write Event Log	LogManager	2009004
Set Dynamic Schema(Metadata Schema)	-	OpenEventDispatcher
Set Dynamic Schema(EventStatus Schema)	-	OpenEventDispatcher
Set Dynamic Schema(Metaframe Schema)	-	Stub::Dispatcher::OpenSDK
Set Dynamic Schema(Metaframe Capability)	-	Stub::Dispatcher::OpenSDK
Event Status Check	-	OpenEventDispatcher
Event Status Changed	-	OpenEventDispatcher
Send Dynamic Metadat	MetadataManager	MetadataManager_0 ~ MetadataManager_#

- Header file and Enum event type

Event type	Header file	Enum type
FTP File Upload	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAppFtpFileUpload
Send Email Notification	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAppSendEmailNoti
Snapshot JPEG	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eAppSnapshotJpeg
Set OSD message	i_p_stream_requester_manager_video.h	IPStreamRequesterManagerVideo::EEventType::eExternalMultiLineOsd
Send Metadata(Event Metadata)	i_metadata_manager.h	IMetadataManager::EEventType::eRequestMetadata
Send Metadata(Frame Metadata)	i_metadata_manager.h	IMetadataManager::EEventType::eRequestMetadata
Write Event Log	i_log_manager.h,life_cycle_manager_interface.h	ILogManager::EEvent::eRemoteDebugMessage
Set Dynamic Schema(Metadata Schema)	i_eventstatus_cgi_dispatcher.h	I_EventstatusCGIDispatcher::EEventType::eMetadataSchema
Set Dynamic Schema(EventStatus Schema)	i_eventstatus_cgi_dispatcher.h	I_EventstatusCGIDispatcher::EEventType::eEventstatusSchema
Set Dynamic Schema(Metaframe Schema)	i_opensdk_cgi_dispatcher.h	I_OpenSDKCGIDispatcher::EEventType::eMetaFrameSchema
Set Dynamic Schema(Metaframe Options)	i_opensdk_cgi_dispatcher.h	I_OpenSDKCGIDispatcher::EEventType::eMetaFrameCapability
Event Status Check	i_eventstatus_cgi_dispatcher.h	I_EventstatusCGIDispatcher::EEventType::eEventStatusCheck
Event Status Changed	i_eventstatus_cgi_dispatcher.h	I_EventstatusCGIDispatcher::EEventType::eEventStatusChanged
Send Dynamic Metadat	p_metadata_manager.h	IMetadataManager::EEventType::eRequestRawMetadata

3.2.1. FTP File Upload

With this event, you can upload files to an FTP server.

- Parameter
  - StringBuffer

Variable	Type	Description
app_id	string	App's ID
file_type	IFtpClient::EFileType	The file type to enum. \{JPEG = 0, XLSX, TXT, PNG}
file_path	string	Path of the file to be transferred to the FTP server.
event_info	string	The event information. (file neme: \{Carmera IP}_\{Time stemp}_*\{Event Info}*_\{Report Name})
report_name	string	The report name. (file neme: \{Carmera IP}_\{Time stemp}_\{Event Info}_*\{Report Name}*)

- Preconditions
  - Upload the file to the application running the cURL command. The upload command is as follows:

```
curl -v --digest -u {Web ID}:{Web PASSWORD} -F DataFile=@{UPLOAD FILE NAME};type=application/octet-stream
"http://{Camera IP}/stw-cgi/opensdk.cgi?msubmenu=apps&action=update&AppID={APP ID}" -H "Expect:"
```

  - Set up an ftp file server.http://\{Camera IP}/wmf/index.html#/setup/event_ftpemail
- Example

CPP

```cpp
void SampleComponent::FTPFileUpload() {
  JsonUtility::JsonDocument document(JsonUtility::Type::kObjectType);
  auto& alloc = document.GetAllocator();

  document.AddMember("app_id", "App_ID", alloc);
  document.AddMember("file_type", "2", alloc);
  document.AddMember("file_path", "../test.txt", alloc);
  document.AddMember("event_info", "info", alloc);
  document.AddMember("report_name", "report", alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  document.Accept(writer);

  SendNoReplyEvent((uint64_t)Component::EReceivers::eOpenPlatformManager,
(int32_t)IPOpenPlatformManager::EAppEventType::eAppFtpFilepuload, 0,
                 new ("Query") SerializableString(strbuf.GetString()));
}
```

3.2.2. Send Email Notification

With this event, you can set metadataframe options.

- Parameter
  - StringBuffer

Variable	Type	Description
app_id	string	App's ID
file_type	IFtpClient::EFileType	The file type to enum. \{NONE = 0, JPEG, XLSX, TXT, PNG}
file_path	string	Path of the file to be transferred to the SMTP server.
subject	string	Subject of mail
event_info	string	The event information. (file neme: \{Carmera IP}_\{Time stemp}_*\{Event Info}*_\{Report Name})
report_name	string	The report name. (file neme: \{Carmera IP}_\{Time stemp}_\{Event Info}_*\{Report Name}*)

- Preconditions
  - Upload the file to the application running the cURL command. The upload command is as follows:

```
curl -v --digest -u {Web ID}:{Web PASSWORD} -F DataFile=@{UPLOAD FILE NAME};type=application/octet-stream
"http://{Camera IP}/stw-cgi/opensdk.cgi?msubmenu=apps&action=update&AppID={APP ID}" -H "Expect:"
```

  - Set up the SMTP server.http://\{Camera IP}/wmf/index.html#/setup/event_ftpemail
- Example

```cpp
void SampleComponent::SendEmailNotification() {
  JsonUtility::JsonDocument document(JsonUtility::Type::kObjectType);
  auto& alloc = document.GetAllocator();

  document.AddMember("app_id", "App_ID", alloc);
  document.AddMember("file_type", "2", alloc);
  document.AddMember("file_path", "../test.txt", alloc);
  document.AddMember("subject", "subject", alloc);
  document.AddMember("event_info", "info", alloc);
  document.AddMember("report_name", "report", alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  document.Accept(writer);

  SendNoReplyEvent((uint64_t)Component::EReceivers::eOpenPlatformManager,
(int32_t)IPOpenPlatformManager::EAppEventType::eAppSendEmailNoti, 0,
                 new ("Query") SerializableString(strbuf.GetString()));
}
```

3.2.3. Snapshot JPEG

With this event, you can saving a raw image from the camera into JPEG format.

- Special note

```
This Event is not supported in 24.01.05 Version of PNM-C34404RQPZ and PNM-C32083RVQ models.
The PNM-C34404RQPZ and PNM-C32083RVQ models will support this event from the next version.
```

- Parameter
  - StringBuffer

Variable	Type	Description
channel	string	Channel of raw videos frames
app_name	string	App's name
jpeg_path	string	The relative path where you want to save the JEPG file.

- Example

```cpp
void SampleComponent::JpegEncode() {
  JsonUtility::JsonDocument document(JsonUtility::Type::kObjectType);
  auto& alloc = document.GetAllocator();

  document.AddMember("channel", "0", alloc);
  document.AddMember("app_name", "App_Name", alloc);
  document.AddMember("jpeg_path", "../test_done.jpg", alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  document.Accept(writer);

  SendNoReplyEvent((uint64_t)Component::EReceivers::eOpenPlatformManager,
(int32_t)IPOpenPlatformManager::EAppEventType::eAppSnapshotJpeg, 0,
                 new ("Query") SerializableString(strbuf.GetString()));
}
```

3.2.4. Set OSD Message

With this event, you can set OSD messages. The message passed will be displayed, along with the camera title. OenApp uses one last index of OSD message. If it overlaps with the camera's OSD, the OpenApp OSD will be displayed. If the OSD is not used in OpenApp or when the app is terminated, the OSD disable event must be sent by setting Enable of the OSD to Off.

- Special note
  - To use the Set OSD Message API, you must check the parameters defined in the Protocol attribute file. The method to check the parameters defined in the Protocol attribute file is as follows.

```
http://<Camera IP>/stw-cgi/attributes.cgi
```

  - The parameters you need to check to use the OSD Message API are as follows:

OSD Option	Parameter	Value
Position of OSD message	PositionX and PositionY	min, max value
	OSDPosition	enum value

OSD Option	Parameter	Value
Font Size of OSD message	CustomFontSize	min, max value
	FontSize	enum value

- To set the Position of OSD Message, if PositionX and PositionY exist, use the min and max values of PositionX and PositionY, and if OSDPosition exists, use the enum value of OSDPosition.
- To set the Font Size of OSD Message, if CustomFontSize exists, use the min and max values of CustomFontSize. If CustomFontSize does not exist, use the enum value of FontSize.

- Parameter
  - MultilineOsdItem

Variable	Type	Description
Enable	bool	true/false
Index	int32_t	Index of osd message(Only 101)
PositionX	int32_t	Position X(min ~ max)
PositionY	int32_t	Position Y(min ~ max)
OSDPosition	string	(For BWC Carmera Only) TopLeft/TopRight/BottomLeft/BottomRight
OSDType	string	Message Type
FontSize	string	Small/Medium/Large
CustomFontSize	int32_t	Font Size(min ~ max)
OSDColor	string	White/Black/Gray
Transparency	string	Off/Low/Medium/High
OSD	string	User's OSD

- Example

CPP

```
    void SampleComponent::SetOSDMessage() {
      //Input datas to MultilineOsdItem
      IPStreamRequesterManagerVideo::MultilineOsdItem recv_data;

      recv_data.Enable = true;
      recv_data.Index = 101;
      recv_data.OSDType = "Title";
      recv_data.OSDColor = "White";
      recv_data.Transparency = "Off";
      recv_data.OSD = "Test OSD!!!";

      // FontSize
      recv_data.FontSize = "Small";
      // CustomFontSize
      recv_data.CustomFontSize = 50;

      // PositionX, PositionY
      recv_data.PositionX = 50;
      recv_data.PositionY = 50;
      // OSDPosition
      recv_data.PositionY = "TopLeft";

      IPStreamRequesterManagerVideo::MultilineExternalOsdDeliveryObject* temp = new ("osd_list")
IPStreamRequesterManagerVideo::MultilineExternalOsdDeliveryObject();
      if (temp == nullptr) {
        printf("[CHECK] SetOsdMessage ERROR: temp is null\n");
        result = false;
        break;
      }
      temp->vMultilineOsd.clear();
      temp->vMultilineOsd.push_back(recv_data);

      SendNoReplyEvent("SRMgrVideo", (int32_t)IPStreamRequesterManagerVideo::EEventType::eExternalMultiLineOsd, 0,
temp);
    }
    ```
```

### 3.2.5. Send Metadata

With this event, you can send the metadata event to MetadataManager in the system container. This event generates and sends Metaeata that follows the ONVIF specification using input data.When an application requests the metadata to be sent to the MetadataManager in the system container, the camera will send it upon every request. However, the interval between the metadata should be over 30 ms to ensure quality of service. For more information about ONVIF Metadata, follow the link: click here (https://www.onvif.org/specs/srv/analytics/ONVIF-VideoAnalytics-Service-Spec-v221.pdf)

- Common Parameter
  - StreamMetadata

| Function | Return | Parameter | Description |
|---|---|---|---|
| set_frame_metadata | void | EventMetadataItem&& event_metadata | Set frame_metadata |
| frame_metadata | FrameMetadataItem& | - | Returns frame_metadata |
| set_ptz_metadata | void | PtzMetadataItem& ptz_metadata | Set ptz_metadata |
```

Function	Return	Parameter	Description
ptz_metadata	PtzMetadataItem&	-	Returns ptz_metadata
set_event_metadata	void	vector& event_metadata	Set event_metadata
add_event_metadata	void	EventMetadataItem&& event_metadata	Add event_metadata
event_metadata	vector&	-	Returns event_metadata
NeedBroadcasting	void	bool broadcast	Set broadcast
SetSyncInfo	void	uint64_t profile_id, uint64_t session_id	Set Sync information
SetSessionID	void	uint64_t session_id	Set session_id
GetSessionID	uint64_t	-	Returns session_id
SetProfileID	void	uint64_t profile_id	Set profile_id
GetProfileID	uint64_t	-	Returns profile_id
HasFrame	bool	-	Returns whether frame_metadata is present or not
HasPtz	bool	-	Returns whether ptz_metadata is present or not
HasEvent	bool	-	Returns whether event_metadata is present or not
IsBroadcastable	bool	-	Check whether it is Broadcastable
IsSyncable	bool	-	Check whether it is Syncable

3.2.5.1. Event Metadata

With this event, you can send event metadata to the camera.

- Parameter
  - EventMetadataItem

Function	Return	Parameter	Description
set_timestamp	void	uint64_t timestamp	Set TimeStamp

Function	Return	Parameter	Description
timestamp	uint64_t	-	Returns TimeStamp
set_topic	void	string&& topic	Set Tocic
topic	string&	-	Returns topic
set_property	void	string&& property	Set property
property	string&	-	Returns property
add_source	void	SimpleItem&& source	Add source
source	vector&	-	Returns source
add_key	void	SimpleItem&& key	Add key
key	vector&	-	Returns key
add_data	void	SimpleItem&& data	Add data
data	vector&	-	Returns data
add_element	void	ElementItem&& element	Add element
element	vector&	-	Returns element

- SimpleItem

Variable	Type	Description
name	string	Simple item's key
value	string	Simple item's Value

- ElementItem

Variable	Type	Description
element_name	string	Element's name
descriptor_name	string	Descriptor's name
element_data	string	Element data
candidate	vector	candidate's struct

- Candidate

Variable	Type	Description
type_name	string	Candidate's Name

Variable	Type	Description
likelihood	float	Candidate's likelihood

- Example

```cpp
void SampleComponent::SendEventMetadata() {
  //Input datas to StreamMetadata
  int channel = 0;
  auto timestamp = std::chrono::duration_cast<std::chrono::milliseconds>
(std::chrono::system_clock::now().time_since_epoch()).count();
  auto metadata = StreamMetadata(channel, timestamp);
  auto event_metadata = EventMetadataItem();

  event_metadata.set_topic("tns1:VideoSource/tnssamsung:" + <App Name>);
  event_metadata.set_timestamp(timestamp);
  event_metadata.add_source({"VideoSourceToken", "VideoSourceToken-0"});
  event_metadata.add_data({"State", "true"});
  //Input datas to Element_item
  EventMetadataItem::ElementItem element_item;
  element_item.element_name = "sample_element_name";
  element_item.descriptor_name = "sample_descriptor_name";
  EventMetadataItem::Candidate candidate;
  candidate.type_name = "sample_type_name";
  candidate.likelihood = 0.94;
  element_item.candidate.push_back(candidate);
  event_metadata.add_element(std::move(element_item));

  metadata.add_event_metadata(std::move(event_metadata));
  metadata.NeedBroadcasting(true);
  SendMetadata(std::move(metadata));
}
```

  - Send Metadata

```cpp
void SampleComponent::SendMetadata(class StreamMetadata&& metadata) {
  auto builder = IPMetadataManager::StreamMetadataRequest::builder();

  builder.set_stream_metadata(std::forward<class StreamMetadata>(metadata));
  auto metadata_request = reinterpret_cast<IPMetadataManager::StreamMetadataRequest*>(builder.build());
  SendNoReplyEvent("MetadataManager", static_cast<int32_t>(IMetadataManager::EEventType::eRequestMetadata), 0,
metadata_request);
}
```

3.2.5.2. Frame Metadata

With this event, you can send frame metadata to the camera.

- Parameter
  - FrameMetadataItem

Function	Return	Parameter	Description
set_timestamp	void	uint64_t timestamp	Set TimeStamp
timestamp	uint64_t	-	Returns TimeStamp
set_resolution	void	int width, int height	Set width and height

Function	Return	Parameter	Description
width	int	-	Returns width
height	int	-	Returns height
set_media_source_token	void	string& token	Set media_source_token
media_source_token	string&	-	Returns media_source_token
set_video_source_token	void	string& token	Set video_source_token
video_source_token	string&	-	Returns video_source_token
set_geolocation	void	bool enable, float lon, float elevation	Set geolocation
geolocation	metadata::common::Geolocation&	-	Returns geolocation
add_object	void	metadata::common::Object&& object	Add object
object	vectormetadata::common::Object&	-	Returns object

- Object

Variable	Type	Description
rule_id	uint32_t	Rule id info
object_id	uint32_t	Object id info
type	MetadataType	Type info
category	ObjectCategory	Category info
rect	Rect	Rect info
likelihood	float	Likelihood info
parent_id	uint32_t	Parent id info
attr_flag	uint32_t	Attribute flag info
attr_num	uint32_t	Attribute number info
attr_class	uint64_t	Attribute class info
timestamp	uint64_t	TimeStamp info

Variable	Type	Description
crop_img_info	Rect	Crop image info
proximate_obj_id	uint32_t	Proximate object id info
proximate_distance	float	Proximate distance info
image_ref	string	Image reference info

- Rect

Variable	Type	Description
sx	int32_t	Starting point of x
sy	int32_t	Starting point of y
ex	int32_t	Ending point of x
ey	int32_t	Ending point of y

- Example

```cpp
void SampleComponent::SendFrameMetadata() {
  auto timestamp = std::chrono::duration_cast<std::chrono::milliseconds>
(std::chrono::system_clock::now().time_since_epoch()).count();
  auto frame_metadata = FrameMetadataItem();
  frame_metadata.set_resolution(3840, 2160);
  frame_metadata.set_timestamp(timestamp);

  metadata::common::Object send_object;
  send_object.object_id = 2; //(uint32_t)obj->get_id();
  send_object.parent_id = 0; //(uint32_t)obj->get_parent_id();
  send_object.timestamp = timestamp;
  send_object.likelihood = (float)0.994629; //(float)obj->get_confidence() / 100.f;
  send_object.category = (metadata::common::ObjectCategory)0;
  send_object.attr_num = 0; //obj->get_attrib_info()->attribute_num;
  send_object.rect.sx = 1685;
  send_object.rect.ex = 1920;
  send_object.rect.sy = 68;
  send_object.rect.ey = 374;
  frame_metadata.add_object(std::move(send_object));

  printf("[CHECK] frame metadata ObjectID[%d], likehood[%f] rect[x:%d,%d y:%d,%d]\n",
          send_object.object_id,
          send_object.likelihood,
          send_object.rect.sx,
          send_object.rect.ex,
          send_object.rect.sy,
          send_object.rect.ey);
  auto metadata = StreamMetadata(GetChannel(), timestamp, Metadata::MetadataType::kAI);
  metadata.set_frame_metadata(std::move(frame_metadata));
  SendMetadata(std::move(metadata));
}
```

- Send Metadata

```cpp
void SampleComponent::SendMetadata(class StreamMetadata&& metadata) {
  auto builder = IPMetadataManager::StreamMetadataRequest::builder();

  builder.set_stream_metadata(std::forward<class StreamMetadata>(metadata));
  auto metadata_request = reinterpret_cast<IPMetadataManager::StreamMetadataRequest*>(builder.build());
  SendNoReplyEvent("MetadataManager", static_cast<int32_t>(IMetadataManager::EEventType::eRequestMetadata), 0,
metadata_request);
}
```

3.2.6. Write Event Log

With this event, you can write event logs to the camera.

- Parameters
  - Log::Log

Variable	Type	Description
_type	Log::LogType	EVENT_LOG
_detail_type	Log::LogDetailType	EVENT_OPENAPP
_parameter	int32_t	-
_time	time_t	-
_string	string	Log
_channel	int32_t	-

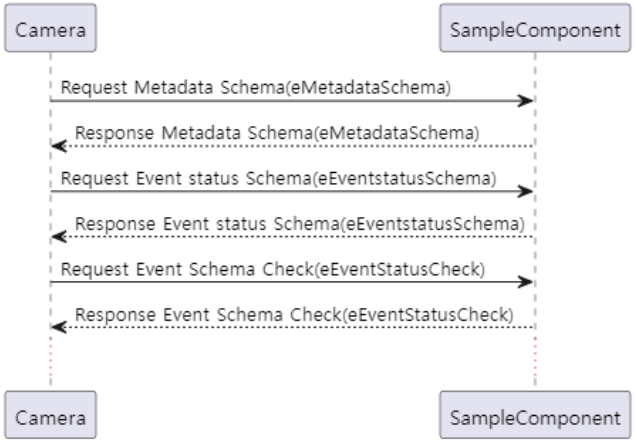
- Example

```cpp
void SampleComponent::SendWriteEventLog() {
  std::string log_message = "Semple App Test Log!"
  auto* log = new Log(Log::LogType::EVENT_LOG, Log::LogDetailType::EVENT_OPENAPP, 0, time(NULL),
String(log_message.c_str()));
  SendNoReplyEvent("LogManager", (int32_t)ILogManager::EEvent::eWrite, 0, log);
}
```

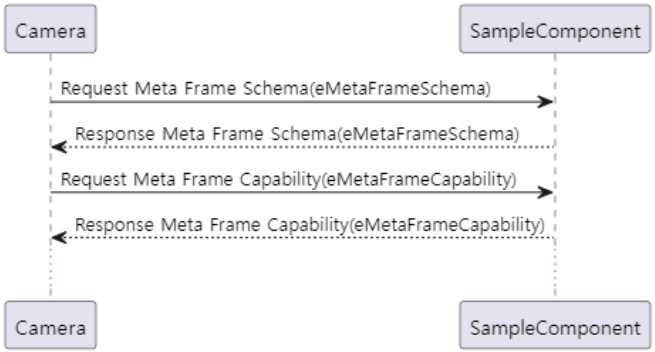
3.2.7. Set Dynamic Schema

With this event, you can set dynamic event schemas to cameras. - For simple True/false event, you should use the event type because there is no detailed event schema. - If you want to push the metadata event, make meta event schema first, and send the event that follows schema you made. (For the metadata event schema, it is recommended that you use ONVIF standard scheme.) - If you send JSON or xml files, check the validation first.

The process of setting up the schema is as shown below. * Metadata Schema & EventStatus Schema



- MetaFrame Scheme & EventStatus Schema



3.2.7.1. Metadata Schema

This event is used to request Metadata Schema from the Camera to the User Component, and to add XML format Metadata Schema from the User Component to the Camera.(RTP)

- Parameters
  - SerializableString

Variable	Type	Description
PROPRIETARY	JSON	Add Proprietary Schema Member
ONVIF	JSON	Add Onvif Schema Member

    - JSON

Variable	Type	Description
EventName	string	Event Metadata Name

Variable	Type	Description
EventTopic	sting	Event Metadata Topic
EventSchema	string	Event Metadata Schema

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  bool result = true;
  switch (event->GetType()) {
    case static_cast<int32_t>(I_EventstatusCGIDispatcher::EEventType::eMetadataSchema): {
      SendMetadataSchema(event);
      break;
    }
  ...
  }
  return result;
}

void SampleComponent::SendMetadataSchema(Event* event) {
  JsonUtility::JsonDocument document(JsonUtility::Type::kObjectType);
  auto alloc = document.GetAllocator();

  std::string meta_schema =
      "<tns1:OpenApp><dynamic_event>"
      "<TestSingleEvent wstop:topic=\"true\"><tt:MessageDescription IsProperty=\"true\">"
      "<tt:Source>"
      "<tt:SimpleItemDescription Name=\"VideoSourceToken\" Type=\"tt:ReferenceToken\"/>"
      "<tt:SimpleItemDescription Name=\"RuleName\" Type=\"xsd:string\"/>"
      "</tt:Source>"
      "<tt:Data>"
      "<tt:SimpleItemDescription Name=\"State\" Type=\"xsd:boolean\"/>"
      "</tt:Data>"
      "</tt:MessageDescription></TestSingleEvent></dynamic_event></tns1:OpenApp>";

  rapidjson::Value prop(rapidjson::kObjectType);
  prop.AddMember("EventName", "OpenSDK.dynamic_event.TestSingleEvent", alloc);        // "OpenSDK." + <AppName> + "."
+ <EventName>
  prop.AddMember("EventTopic", "tns1:OpenApp/dynamic_event/TestSingleEvent", alloc);  // "tns1:OpenApp/" + <AppName>
+ "/" + <EventName>
  prop.AddMember("EventSchema", meta_schema, alloc);

  rapidjson::Value onvif(rapidjson::kObjectType);
  onvif.AddMember("EventName", "OpenSDK.dynamic_event.TestSingleEvent", alloc);
  onvif.AddMember("EventTopic", "tns1:OpenApp/dynamic_event/TestSingleEvent", alloc);
  onvif.AddMember("EventSchema", meta_schema, alloc);

  document.AddMember("PROPRIETARY", prop, alloc);
  document.AddMember("ONVIF", onvif, alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  document.Accept(writer);
  Schema_infos_["metadata_schema"] = strbuf.GetString();
  SendNoReplyEvent("OpenEventDispatcher", event->GetType(), 0, new ("MetadataSchema") String(strbuf.GetString()));
}
```

- To check whether the metadata schema has been added, follow the link below:

```text
http://<Camera IP>/stw-cgi/eventstatus.cgi?msubmenu=metadataschema&action=view
```

3.2.7.2. EventStatus Schema

This event is used to request EventStatus Schema from the Camera to the User Component, and to add EventStatus Schema from the User Component to the Camera.(SUNAPI Monitor)

- Parameters
  - SerializableString

Variable	Type	Description
Schema	string	Event Status Schema String

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  bool result = true;
  switch (event->GetType()) {
    case static_cast<int32_t>(I_EventstatusCGIDispatcher::EEventType::eEventstatusSchema): {
      SendMetadataSchema(event);
      break;
    }
  ...
  }
  return result;
}

void SampleComponent::SendEventStatusSchema(Event* event) {
  std::string json_schema   = "{\"JSON\": {\"type\": \"object\",\"properties\": {\"Time\": {\"type\":
\"string\"},\"EventName\": {\"enum\": [\"OpenSDK.dynamic_event.TestSingleEvent\"]},\"Source\": {\"type\":
\"object\",\"properties\": {\"Channel\": {\"type\": \"number\"},\"AppName\": {\"type\": \"string\"},\"AppEvent\":
{\"type\": \"string\"},\"AppID\": {\"type\": \"string\"},\"Type\": {\"enum\": [\"Event\"]},\"RuleIndex\": {\"type\":
\"number\"},\"VideoSourceToken\": {\"type\": \"string\"},\"RuleName\": {\"type\": \"string\"}}},\"Data\": {\"type\":
\"object\",\"properties\": {\"State\": {\"type\": \"boolean\"}}}}},\"TEXT\": {\"SCHEME\":
\"Name=OpenSDK.dynamic_event.TestSingleEvent\\nSchema.1.Name=Channel.
<int>.OpenSDK.dynamic_event.TestSingleEvent\\nSchema.1.Value=<boolean>\\nSchema.2.Name=Channel.
<int>.OpenSDK.dynamic_event.TestSingleEvent.<int>.VideoSourceToken\\nSchema.2.Value=<string>\\nSchema.3.Name=Channel.
<int>.OpenSDK.dynamic_event.TestSingleEvent.<int>.RuleName\\nSchema.3.Value=<string>\\nSchema.4.Name=Channel.
<int>.OpenSDK.dynamic_event.TestSingleEvent.<int>.State\\nSchema.4.Value=<boolean>\"}}");

  Schema_infos_["event_status_schema"] = query->GetString();
  SendNoReplyEvent("OpenEventDispatcher", event->GetType(), 0, query);
}
```

- To check whether the eventstatus schema has been added, follow the link below:

```text
http://<Camera IP>/stw-cgi/eventstatus.cgi?msubmenu=eventstatusschema&action=view
```

3.2.8. MetaFrame Scheme

This event is used to request MetaFrame Schema from the Camera to the User Component, and to add MetaFrame Schema from the User Component to the Camera.

- Parameters
  - SerializableString

Variable	Type	Description
Schema	string	Frame metadata schema String

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  bool result = true;
  switch (event->GetType()) {
    case static_cast<int32_t>(I_OpenSDKCGIDispatcher::EEventType::eMetaFrameSchema): {
      SendMetaFrameSchema(event);
      break;
    }
  ...
  }
  return result;
}

void SampleComponent::SendMetaFrameSchema(Event* event) {
  if (!event->IsReply()) {
    std::string meta_capa =
        "{"
        "\"AppID\": \"dynamic_event\","
        "\"Schema\": \"\","
        "\"Encoding\": \"base64\""
        "}";
    Schema_infos_["meta_frame_schema"] = meta_capa;
    SendNoReplyEvent("Stub::Dispatcher::OpenSDK", (int32_t)I_OpenSDKCGIDispatcher::EEventType::eMetaFrameSchema, 0,
                     new ("") SerializableString(meta_capa.c_str()));
  }
}
```

- To check whether the frame metadata schema has been set, follow the link below:

```text
http://<Camera IP>/stw-cgi/opensdk.cgi?msubmenu=metaframeschema&action=view
```

3.2.9. MetaFrame Capability

This event is used to request MetaFrame Capability from the Camera to the User Component, and to add MetaFrame Capability from the User Component to the Camera.

- Parameters
  - SerializableString

Variable	Type	Description
Schema	string	MetaFrame Capability String

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  bool result = true;
  switch (event->GetType()) {
    case static_cast<int32_t>(I_OpenSDKCGIDispatcher::EEventType::eMetaFrameCapability): {
      SendMetaFrameCapability(event);
      break;
    }
  ...
  }
  return result;
}

void SampleComponent::SendMetaFrameCapability(Event* event) {
  if (!event->IsReply()) {
    std::string meta_capa =
      "{"
      "\"AppID\": \"dynamic_event\""
      "\","
      "\"Capabilities\": ["
      "{"
      "\"xpath\": \"//tt:VideoAnalytics/tt:Frame/tt:Object/tt:Appearance/tt:Class/tt:Type\","
      "\"type\": \"xs:string\","
      "\"enum\": [\"Face\",\"Human\",\"Vehicle\",\"LicensePlate\",\"Head\",\"Unknown\"]"
      "},"
      "{"
      "\"xpath\": \"//tt:VideoAnalytics/tt:Frame/tt:Object/tt:Appearance/tt:Class/tt:Type/@Likelihood\","
      "\"type\": \"xs:float\","
      "\"minimum\": 0.0,"
      "\"maximum\": 1.0"
      ...
      "}";
    Schema_infos_["meta_frame_capability"] = meta_capa;
    SendNoReplyEvent("Stub::Dispatcher::OpenSDK", (int32_t)I_OpenSDKCGIDispatcher::EEventType::eMetaFrameCapability,
0,
                     new ("") SerializableString(meta_capa.c_str()));
  }
}
```

- To check whether the MetaFrame Capability have been added, follow the link below:

```text
http://<Camera IP>/stw-cgi/opensdk.cgi?msubmenu=metaframecapability&action=view
```

3.2.10. Event Status Check

This event is used by the camera to check the initial Event State of the User Component. When a User Component receives the Event Status Check event, must send the Event State to the camera using the registered Event Status Schema.

- Parameters
  - SerializableString

Variable	Type	Description
data	string	Event Status data you want to send.

- Example

CPP

```cpp
void SampleComponent::NotifyChannelEvent() {
  JsonUtility::JsonDocument document(JsonUtility::Type::kObjectType);
  auto& alloc = document.GetAllocator();

  auto channel_event_object = JsonUtility::ValueType(JsonUtility::Type::kObjectType);
  JsonUtility::set(channel_event_object, "Channel", 0, alloc);
  JsonUtility::set(channel_event_object, "State", true, alloc);
  {
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>
(std::chrono::system_clock::now().time_since_epoch());
    auto time_point = std::chrono::high_resolution_clock::time_point(ms);
    auto time = std::chrono::high_resolution_clock::to_time_t(time_point);
    auto conv_time = localtime(&time);
    if (conv_time == nullptr) {
      return;
    }
    time_t time_val = mktime(conv_time);

    std::stringstream ss("");
    ss << std::put_time(gmtime(&time_val), "%FT%T.") << std::setfill('0') << std::setw(3) << ms.count() % 1000 <<
std::put_time(conv_time, "%z");

    auto time_str = ss.str();
    time_str.insert(time_str.size() - 2, ":");
    JsonUtility::set(channel_event_object, "Time", time_str, alloc);
  }

  JsonUtility::set(channel_event_object, "EventName", "OpenSDK.dynamic_event.TestSingleEvent", alloc);

  auto source_object = JsonUtility::ValueType(JsonUtility::Type::kObjectType);
  {
    JsonUtility::set(source_object, "Channel", 0, alloc);
    JsonUtility::set(source_object, "AppName", "dynamic_event", alloc);
    JsonUtility::set(source_object, "AppID", "dynamic_event", alloc);
    JsonUtility::set(source_object, "AppEvent", "TestSingleEvent", alloc);
    JsonUtility::set(source_object, "Type", "Event", alloc);
    JsonUtility::set(source_object, "RuleIndex", 1, alloc);
    std::string video_source_token = "vs-" + std::to_string(0);
    JsonUtility::set(source_object, "VideoSourceToken", video_source_token, alloc);
    JsonUtility::set(source_object, "RuleName", "Rule-1", alloc);
  }
  channel_event_object.AddMember("Source", source_object, alloc);

  auto data_object = JsonUtility::ValueType(JsonUtility::Type::kObjectType);
  { JsonUtility::set(data_object, "State", true, alloc); }
  channel_event_object.AddMember("Data", data_object, alloc);

  document.AddMember("ChannelEvent", channel_event_object, alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  document.Accept(writer);
  std::string str_out = strbuf.GetString();
  try {
    SendNoReplyEvent("OpenEventDispatcher", (int32_t)I_EventstatusCGIDispatcher::EEventType::eEventStatusCheck, 0,
new ("") String(str_out.c_str()));
  } catch (Exception& e) {
  }
}
```

3.2.11. Event Status Changed

This event send the Event State to the camera when the Event Status of the User Component changes. Event Status is must be created with the registered Event Status Schema structure to the camera.

- Parameters
  - SerializableString

Variable	Type	Description
data	string	Changed Event Status data you want to send.

- Example

CPP

```cpp
void SampleComponent::NotifyChannelEvent(Event* event) {
  if (event == nullptr || event->IsReply()) {
    return;
  }

  JsonUtility::JsonDocument document(JsonUtility::Type::kObjectType);
  auto& alloc = document.GetAllocator();

  auto channel_event_object = JsonUtility::ValueType(JsonUtility::Type::kObjectType);
  JsonUtility::set(channel_event_object, "Channel", 0, alloc);
  JsonUtility::set(channel_event_object, "State", true, alloc);
  {
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>
(std::chrono::milliseconds(std::chrono::system_clock::now().time_since_epoch()));
    auto time_point = std::chrono::high_resolution_clock::time_point(ms);
    auto time = std::chrono::high_resolution_clock::to_time_t(time_point);
    auto conv_time = localtime(&time);
    if (conv_time == nullptr) {
      return;
    }
    time_t time_val = mktime(conv_time);

    std::stringstream ss("");
    ss << std::put_time(gmtime(&time_val), "%FT%T.") << std::setfill('0') << std::setw(3) << ms.count() % 1000 <<
std::put_time(conv_time, "%z");

    auto time_str = ss.str();
    time_str.insert(time_str.size() - 2, ":");
    JsonUtility::set(channel_event_object, "Time", time_str, alloc);
  }

  JsonUtility::set(channel_event_object, "EventName", "OpenSDK.WiseAI.TestSingleEvent", alloc);

  auto source_object = JsonUtility::ValueType(JsonUtility::Type::kObjectType);
  {
    JsonUtility::set(source_object, "Channel", 0, alloc);
    JsonUtility::set(source_object, "AppName", "WiseAI", alloc);
    JsonUtility::set(source_object, "AppID", "WiseAI", alloc);
    JsonUtility::set(source_object, "AppEvent", "TestSingleEvent", alloc);
    JsonUtility::set(source_object, "Type", "Event", alloc);
    JsonUtility::set(source_object, "RuleIndex", 1, alloc);
    std::string video_source_token = "vs-" + std::to_string(0);
    JsonUtility::set(source_object, "VideoSourceToken", video_source_token, alloc);
    JsonUtility::set(source_object, "RuleName", "Rule-1", alloc);
  }
  channel_event_object.AddMember("Source", source_object, alloc);

  auto data_object = JsonUtility::ValueType(JsonUtility::Type::kObjectType);
  { JsonUtility::set(data_object, "State", true, alloc); }
  channel_event_object.AddMember("Data", data_object, alloc);

  document.AddMember("ChannelEvent", channel_event_object, alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  document.Accept(writer);
  std::string str_out = strbuf.GetString();

  try {
    SendNoReplyEvent("OpenEventDispatcher", (int32_t)I_EventstatusCGIDispatcher::EEventType::eEventStatusChanged , 0,
new ("") String(str_out.c_str()));
  } catch (Exception& e) {
  }
}
```

3.2.12. Send Dynamic Metadat

Same as "Send Metadata", but you must create the metadata yourself. Use only "Send Metadata" or "Send Dynamic Metadata".

- Parameters
  - SerializableString

Variable	Type	Description
data	string	Dynamic data you want to send.

- Example

```cpp
void SampleComponent::SendDynamicEvent() {
  int channel = 0;
  auto now = std::chrono::system_clock::now();
  auto timestamp = std::chrono::duration_cast<std::chrono::milliseconds>(now.time_since_epoch()).count();

  auto metadata = StringMetadata(channel, timestamp);

  std::string data = "<?xml version="1.0" encoding=\"UTF-8\"?><tt:MetadataStream
xmlns:tt=\"http://www.onvif.org/ver10/schema\" xmlns:ttr=\"https://www.onvif.org/ver20/analytics/radiometry\"
xmlns:wsnt=\"http://docs.oasis-open.org/wsn/b-2\" xmlns:tns1=\"http://www.onvif.org/ver10/topics\"
xmlns:tnssamsung=\"http://www.samsungcctv.com/2011/event/topics\"><tt:Event><wsnt:NotificationMessage><wsnt:Topic
Dialect=\"http://www.onvif.org/ver10/tev/topicExpression/ConcreteSet\">tns1:OpenApp/dynamic_event/TestSingleEvent</ws
nt:Topic><wsnt:Message><tt:Message UtcTime=\"2024-04-05T00:02:21.782Z\"><tt:Source><tt:SimpleItem
Name=\"VideoSourceToken\" Value=\"vs-0\"/><tt:SimpleItem Name=\"RuleName\" Value=\"Rule-1\"/></tt:Source><tt:Key>
</tt:Key><tt:Data><tt:SimpleItem Name=\"State\" Value=\"false\"/></tt:Data></tt:Message></wsnt:Message>
</wsnt:NotificationMessage></tt:Event></tt:MetadataStream>";

  metadata.Set(data);

  auto req = new ("MetadataRequest") IPMetadataManager::StringMetadataRequest();
  req->SetStringMetadata(std::move(metadata));

  SendNoReplyEvent("MetadataManager", (int32_t)IMetadataManager::EEventType::eRequestRawMetadata, 0, req);
  break;
}
```

3.3. SD Card Events

SD-card events allow you to use SD card for apps. To receive sd-card events, make sure that the app_manifest.json file inside the app has the "SDCard" in "Permission" array. If you add the permission and intall the app, a popup will appear during the installation process asking whether to grant the SDCard permission.

- **app_manifest.json**

```
{
  "AppName": "sample",
  "AppVersion": "1.0",
  ...
  "Permission": [
    "SDCard"
  ]
}
```

- **ReceiverNames** Registration

Event type	SymbolName	RealName
Get SD Card Status	OpenPlatform	EComponents::eOpenPlatform
Start SD Card	OpenPlatform	EComponents::eOpenPlatform
Stop SD Card	OpenPlatform	EComponents::eOpenPlatform
Format SD Card	OpenPlatform	EComponents::eOpenPlatform
Set Exclusive Mode	OpenPlatform	EComponents::eOpenPlatform
Get Exclusive Mode Status	OpenPlatform	EComponents::eOpenPlatform

- Header file and Enum event type

Event type	Header file	Enum type
Get SD Card Status	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eGetSDCardStatus
Start SD Card	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eStartSDCard
Stop SD Card	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eStopSDCard
Format SD Card	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eFormatSDCard
Set Exclusive Mode	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eExclusiveMode
Get Exclusive Mode Status	i_p_open_platform_manager.h	IPOpenPlatformManager::EAppEventType::eGetExclusiveModeStatus

3.3.1. Get SD Card Status

With this event, you can retrieve the status of SD Card slots. The Slot index, SD Card presence, and mount status can be checked.

- Parameters
  None.
- Replied Result
  You can get detecting event information in JSON format. See the explanation of each key and example results below:
  - SerializableString

JSON key name	Type	Description
SDCardSlots	json array	Status information for each slot
SlotIndex	int	Slot index
IsPresent	boolean	true if it is present, false if not.
IsMounted	boolean	true if it is mounted, false if not.

```JSON
"SDCardSlots": [
  {
    "SlotIndex": 0,
    "IsPresent": true,
    "IsMounted": true
  },
  {
    "SlotIndex": 1,
    "IsPresent": true,
    "IsMounted": false
  }
]
```

- Example
  See 'sdcard_output_event' sample application.

```CPP
void SampleComponent::GetSDCardStatus() {
  SendReplyEvent("OpenPlatform", (int)IPOpenPlatformManager::EAppEventType::eGetSDCardStatus);
}

void SampleComponent::ParseSDCardStatusResult(Event* event) {
  auto ret = eventToArgumentBuffer(event);
  SerializableString param;
  param.DeserializeBaseObject(&param, ret);

  JsonUtility::JsonDocument doc;
  doc.Parse(param.GetString());
  if (doc.HasMember("SDCardSlots") && doc["SDCardSlots"].IsArray()) {
    for (auto &itr : doc["SDCardSlots"].GetArray()) {
      int slot_index;
      bool is_present, is_mounted;
      JsonUtility::get(itr, "SlotIndex", slot_index);
      JsonUtility::get(itr, "IsPresent", is_present);
      JsonUtility::get(itr, "IsMounted", is_mounted);

      printf("SlotIndex:%d IsPresent:%d, IsMounted:%d \n", slot_index, is_present, is_mounted);
    }
  }
}
```

3.3.2. Start SD Card

With this event, you can perform the mount operation, the process of recognizing the SD card on the device and configuring it to read and write data.

- Caution
  After using this event, do not enable the Record function on the web page. This may lead to malfunction. If you enable the Record function before using this event, the mount operation will take a maximum of 20 seconds.
- Parameters
  - AppMsgParam

Variable	Type	Description
app_name_	String	app name

Variable	Type	Description
message_	String	json format message. see **Message** below.

- Message

key	Type	Value
SDCard	int	0 : 1st SD card, 1 : 2nd SD card

- Preconditions

  Insert the SD card into the slot of the device where you want to install the app.

- Replied Result
  - SerializableString

Type	Value
String	result. if successful, it replies the folder path accessible by the app, and if it fails, it replies "Failure".

- Example
  See 'sdcard_output_event' sample application.

```cpp
void SampleComponent::StartSDCard() {
  JsonUtility::JsonDocument req(JsonUtility::Type::kObjectType);
  auto& alloc = req.GetAllocator();

  JsonUtility::set(req, "SDCard", 0, alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  req.Accept(writer);

  auto* param = new ("AppMsgParam") openplatform::AppMsgParam(app_id, strbuf.GetString());
  SendReplyEvent("OpenPlatform", (int)IPOpenPlatformManager::EAppEventType::eStartSDCard, 0, param);
}

bool SampleComponent::StardSDCardResult(Event* event) {
  auto ret = eventToArgumentBuffer(event);
  SerializableString param;
  param.DeserializeBaseObject(&param, ret);

  std::string sdcard_app_path = param.GetString.c_str();
}
```

3.3.3. Stop SD Card

With this event, you can perform the 'umount' operation that is responsible for unmounting and disconnecting the SD card from the device.

- Parameters
  - AppMsgParam

variable	type	description
app_name_	String	app name
message_	String	json format message. see **Message** below

- Message

key	type	value
SDCard	int	0 : 1st SD card, 1 : 2nd SD card

- Preconditions

  Insert the SD card into the slot of the device where you want to install the app.

- Replied Result
  - SerializableString

Type	Value
String	result. if successful, it replies "Success", and if it fails, it replies "Failure".

- Example
  See 'sdcard_output_event' sample application.

```cpp
void SampleComponent::StopSDCard() {
  JsonUtility::JsonDocument req(JsonUtility::Type::kObjectType);
  auto& alloc = req.GetAllocator();

  JsonUtility::set(req, "SDCard", 0, alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  req.Accept(writer);

  auto* param = new ("AppMsgParam") openplatform::AppMsgParam(app_id, strbuf.GetString());
  SendReplyEvent("OpenPlatform", (int)IPOpenPlatformManager::EAppEventType::eStopSDCard, 0, param);
}

bool SampleComponent::StopSDCardResult(Event* event) {
  auto ret = eventToArgumentBuffer(event);
  SerializableString param;
  param.DeserializeBaseObject(&param, ret);

  std::string stop_result = param.GetString.c_str();
}
```

3.3.4. Format SD Card

With this event, you can erase all existing data on the SD card and set up a new file system structure.

- Parameters
  - AppMsgParam

variable	type	description
app_name_	String	app name
message_	String	json format message. see 'Message' below

- Message

key	type	value
SDCard	int	0 : 1st SD card, 1 : 2nd SD card

- Preconditions
  Insert the SD card into the slot of the device where you want to install the app.
- Replied Result
  - SerializableString

Type	Value
String	result. if successful, it replies "Success", and if it fails, it replies "Failure".

- Example

  See 'sdcard_output_event' sample application.

```cpp
void SampleComponent::FormatSDCard() {
  JsonUtility::JsonDocument req(JsonUtility::Type::kObjectType);
  auto& alloc = req.GetAllocator();

  JsonUtility::set(req, "SDCard", 0, alloc);
  JsonUtility::set(req, "Filesystem", "EXT4", alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  req.Accept(writer);

  auto* param = new ("AppMsgParam") openplatform::AppMsgParam(app_id, strbuf.GetString());
  SendReplyEvent("OpenPlatform", (int)IPOpenPlatformManager::EAppEventType::eFormatSDCard, 0, param);
}

bool SampleComponent::FormatSDCardResult(Event* event) {
  auto ret = eventToArgumentBuffer(event);
  SerializableString param;
  param.DeserializeBaseObject(&param, ret);

  std::string format_result = param.GetString.c_str();
}
```

3.3.5. Set Exclusive Mode

With this event, you can set whether to use the SD card in exclusive mode or not. Exclusive mode indicates that when a device has numerous SD card slots, only one SD card is used for apps.

- Parameters

- AppMsgParam

variable	type	description
app_name_	String	app name
message_	String	json format message. see **Message** below

- Message

key	type	value
use	bool	whether to use exclusive mode or not. Allowed values: [true, false]

- Preconditions
  Insert the SD card into the slot of the device where you want to install the app.
- Example
  See 'sdcard_output_event' sample application.

```cpp
void SampleComponent::SetExclusiveMode() {
  JsonUtility::JsonDocument req(JsonUtility::Type::kObjectType);
  auto& alloc = req.GetAllocator();

  JsonUtility::set(req, "use", true, alloc);

  rapidjson::StringBuffer strbuf;
  rapidjson::Writer<rapidjson::StringBuffer> writer(strbuf);
  req.Accept(writer);

  auto* param = new ("AppMsgParam") openplatform::AppMsgParam(app_id, strbuf.GetString());
  SendNoReplyEvent("OpenPlatform", (int)IPOpenPlatformManager::EAppEventType::eExclusiveMode, 0, param);
}
```

3.3.6. Get Exclusive Mode Status

- Parameters
  - AppMsgParam

variable	type	description
app_name_	String	app name
message_	String	empty string. you don't have to fill in.

- Replied Result
  - AppMsgParam

variable	type	description
app_name_	String	app name

variable	type	description
message_	String	json format message. see 'Message' below

- Message

key	type	value
status	int	0 : Normal Mode 1 : Exclusive Mode 2: Changing Mode

- Preconditions
  Insert the SD card into the slot of the device where you want to install the app.
- Example

  See 'sdcard_output_event' sample application.

```cpp
void SampleComponent::GetExclusiveMode() {
  auto* param = new ("AppMsgParam") AppMsgParam(app_id, "");
  SendReplyEvent("OpenPlatform", (int)IPOpenPlatformManager::EAppEventType::eGetExclusiveModeStatus, 0, param,
(int)IPOpenPlatformManager::EAppEventType::eGetExclusiveModeStatusDone);
}
...
void SampleComponent::ProcessAEvent(Event* event) {
  ...
  case static_cast<int32_t>(IPOpenPlatformManager::EAppEventType::eGetExclusiveModeStatusDone): {
    auto blob = event->GetBlobArgument();
    auto* data = blob.GetRawData();
    if (data != nullptr) {
      std::pair<std::variant<BaseObject*, char*>, uint64_t> ret((char*)blob.GetRawData(), blob.GetSize());
      AppMsgParam param;
      param.DeserializeBaseObject(&param, ret);

      auto message = param.GetMessage();

      JsonUtility::JsonDocument doc;
      doc.Parse(message);

      if(doc.HasMember("status")) {
        int exclusive_mode_ = doc["status"].GetInt();
        printf("Exclusive mode : %d\n", exclusive_mode_);
      }
    }
    break;
  }
  ...
}
```

4. App Support Package

4.1. Network Manager

You can use the network function by exchanging network events with Network Manager. Network Manager is a tool for managing and controlling network communication. Network Manager can be used as a server or client according to a socket type, an IP address, and a port number you designated. When used as a server, it provides a notification when a new client is connected or

disconnected, and it receives and informs the data sent by the client. Events that can be exchanged with Network Manager are as follows:

4.1.1. Sendable Events

4.1.2. Receivable Events

- **SourceNames** Registration

Event type	Source	Group name
New Client Connected	AppNetworkManager	NetworkEvent
Client Disconnected	AppNetworkManager	NetworkEvent
Client Data	AppNetworkManager	NetworkEvent
Server Data	AppNetworkManager	NetworkEvent

- **ReceiverNames** Registration

Event type	Receiver names
Start Service	AppNetworkManager
Stop Service	AppNetworkManager
Send Data	AppNetworkManager
Close Client	AppNetworkManager

- **Header** and **Event Type**
  Include the header file in the source code of the component you want to exchange events with Network Manager.

Event type	Header file	Event type (enum)
Start Service	i_app_network_manager.h	IAppNetworkManager::EEventType::eStartService
Stop Service	i_app_network_manager.h	IAppNetworkManager::EEventType::eStopService
Send Data	i_app_network_manager.h	IAppNetworkManager::EEventType::eSendData
Close Client	i_app_network_manager.h	IAppNetworkManager::EEventType::eCloseClient
New Client Connected	i_app_network_manager.h	IAppNetworkManager::EEventType::eNewClientConnected
Client Disconnected	i_app_network_manager.h	IAppNetworkManager::EEventType::eClientDisconnected
Client Data	i_app_network_manager.h	IAppNetworkManager::EEventType::eClientData
Server Data	i_app_network_manager.h	IAppNetworkManager::EEventType::eServerData

4.1.3. Sendable Events

4.1.3.1. Start Service

With this event, you can request the network manager to start the network service. To that end, you need information, such as the service type, socket type, IP address, and port number. This allows the client or server to be executed. Once the service is started, you cannot request again until it's stopped.

- Parameters
  - NetworkConfig

Variable	Type	Description
service type	Enum1. EServiceType	Run as a server or client
socket type	Enum2. ESocketType	Type of socket
IP	string	Address of the server to connect to. Required only when server type is client.
Port	int	Port number of the server or client to connect to

  - Enum 1. EServiceType

```cpp
enum class EServiceType { eServer, eClient };
```

  - Enum 2. ESocketType

```cpp
enum class ESocketType { eTCP, eUDP, eMulticast };
```

- Example

1. Start Client.

```cpp
auto* arg =
 new ("NetworkConfig") NetworkConfig((int)IAppNetworkManager::EServiceType::eClient,
(int)IAppNetworkManager::ESocketType::eTCP, "127.0.0.1", 8080);
SendNoReplyEvent("AppNetworkManager", (int)IAppNetworkManager::EEventType::eStartService, 0, arg);
```

2. Start Server.

```cpp
auto* arg =
 new ("NetworkConfig") NetworkConfig((int)IAppNetworkManager::EServiceType::eServer,
(int)IAppNetworkManager::ESocketType::eTCP, "", 8080);
SendNoReplyEvent("AppNetworkManager", (int)IAppNetworkManager::EEventType::eStartService, 0, arg);
```

4.1.3.2. Stop Service

With this event, you can request Network Manager to terminate the network service. The request is completed successfully only when the service is already running.

- Parameters
  None.
- Example

```cpp
SendNoReplyEvent("AppNetworkManager", (int)IAppNetworkManager::EEventType::eStopService);
```

4.1.3.3. Send Data

With this event, you can request Network Manager to transmit data. Parameters vary depending on which service was requested in advance. If it is a client, no parameters are required because it transfers data to the Server. However, if it is a server, a client ID is required as a parameter because it transmits data to one of the clients. The client ID will be notified as an event by the network manager whenever a new client is connected. (Refer to New Client Connected.)

- Parameters
  - NetworkBufferData

Variable	Type	Description
data	char*	Data to be transferred
data size	int	Data length
Client id	int	If the network service is running as a server, choose which client to send it to

- Example
  1. After running the server, send "hello world" data to the client with client ID.

```cpp
const char* message = "hello world!";
auto* data = new ("NetworkBufferData") NetworkBufferData(message, strlen(message), client_id);
SendNoReplyEvent("AppNetworkManager", (int)IAppNetworkManager::EEventType::eSendData, 0, data);
```

  2. After running the client, send "hello world" data to the server.

```cpp
const char* message = "hello world!";
auto* data = new ("NetworkBufferData") NetworkBufferData(message, strlen(message));
SendNoReplyEvent("AppNetworkManager", (int)IAppNetworkManager::EEventType::eSendData, 0, data);
```

4.1.3.4. Close Client

With this event, you can request, if necessary, that the connected client be terminated while running the network service as the server. The client ID parameter determines which client to close. The client ID will be notified as an event by Network Manager whenever a new client is connected. (Refer to New Client Connected.)

- Parameters

Variable	Type	Description
client id	int	Client ID to close

- Example

```cpp
SendNoReplyEvent("AppNetworkManager", (int)IAppNetworkManager::EEventType::eCloseClient, client_id);
```

4.1.4. Receivable Events

4.1.4.1. New Client Connected

After running the network service as a server, you can receive a client ID as an event when a new client is connected to the server.

- Parameters

Variable	Type	Description
client id	int	newly connected client id

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int>(IAppNetworkManager::EEventType::eNewClientConnected): {
      int arg = event->GetArgument();
      printf("Client ID is %d", arg);
      break;
    }
    ...
  }
  ...
}
```

4.1.4.2. Client Disconnected

After running the network service as a server, you can receive a client ID as an event when the client connected to the server is disconnected.

- Parameters

Variable	Type	Description
client id	int	Disconnected client id

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int>(IAppNetworkManager::EEventType::eClientDisconnected): {
      int arg = event->GetArgument();
      std::cout << "Client ID is " << arg << std::endl;
      break;
    }
    ...
  }
  ...
}
```

4.1.4.3. Client Data

After running the network service as a client, you can receive the data received by the client as an event when the server transmits data to the client.

- Parameters
  - NetworkBufferData

Variable	Type	Description
data	char*	Data received from the server
data size	int	Data length
Client id	int	-1 (It's from server, not client)

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int>(IAppNetworkManager::EEventType::eClientData): {
      auto* data = reinterpret_cast<NetworkBufferData*>(event->GetBaseObjectArgument());
      if (data == nullptr) {
        std::cout << "data is nullptr" << std::endl;
        break;
      }
      std::cout << data->buffer() << std::endl;
      std::cout << data->buffer_size() << std::endl;
      std::cout << data->client_id() << std::endl;
      break;
    }
    ...
  }
  ...
}
```

4.1.4.4. Server Data

After running the network service as a client, you can receive the data received by the client as an event when the server transmits data to the Client.

- Parameters
  - NetworkBufferData

Variable	Type	Description
data	char*	Data received from the server
data size	int	Data length
Client id	int	-1 (It's from server, not client)

- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case static_cast<int>(IAppNetworkManager::EEventType::eServerData): {
      auto* data = reinterpret_cast<NetworkBufferData*>(event->GetBaseObjectArgument());
      if (data == nullptr) {
        std::cout << "data is nullptr" << std::endl;
        break;
      }
      std::cout << data->buffer() << std::endl;
      std::cout << data->buffer_size() << std::endl;
      break;
    }
    ...
  }
  ...
}
```

4.2. SUNAPI Requester

You can send SUNAPI requests from the application through the SUNAPI Requester. You can also receive the results of the request as an event. For more information on how to use SUNAPI, contact us. We will provide you with the **SUNAPI** document that includes the information.

3.2.1 Sendable Events

3.2.2 Receivable Events

- **SourceNames** Registration

Event type	Source (#:max channel)	Group name
Ready for Use SUNAPI	SunapiRequester	ReadyForUse

- **ReceiverNames** Registration

Event type	Receiver names
Request SUNAPI	SunapiRequester
SUNAPI Response	X (Unnecessary)

- **Header** and **Event Type**

Event type	Header file	Event type (enum)
Request SUNAPI	i_sunapi_reqeuster.h	ISunapiRequester::EEventType::eRequestSunapi
SUNAPI Response	i_sunapi_reqeuster.h	ISunapiRequester::EEventType::eRequestSunapiDone
Ready for Use SUNAPI	i_sunapi_requetser.h	ISunapiRequester::EEventType::eReadyForUse

4.2.1. Sendable Events

4.2.1.1. Request SUNAPI

- Parameters
  - String

Variable	Type	Description
request	string	SUNAPI request

- Sendable SUNAPI request example (For more information on how to use SUNAPI, contact us. We will provide you with the SUNAPI document that includes the information.)

Description	SUNAPI Request
Get video source	/stw-cgi/media.cgi?msubmenu=videosource&action=view
Set video source	/stw-cgi/media.cgi?videosource&action=set&Channel=0&VideoType=CVI&VideoMode=Auto (changeable)
Get profile info	/stw-cgi/media.cgi?msubmenu=videoprofile&action=view&ShowAll=True
Get profile config	/stw-cgi/media.cgi?msubmenu=videoprofile&action=view&Profile=1 (changeable)
Add profile	/stw-cgi/media.cgi?msubmenu=videoprofile&action=add&Name=Test&EncodingType=MJPEG (changeable)
Delete profile	/stw-cgi/media.cgi?msubmenu=videoprofile&action=remove&Channel=0&Profile=7 (changeable)
Get bitrate limit	/stw-cgi/media.cgi?msubmenu=videocodecinfo&action=view&EncodingType=MJPEG (changeable)
Get framerate limit	/stw-cgi/media.cgi?msubmenu=videocodecinfo&action=view&EncodingType=MJPEG (changeable)
Get support resolution	/stw-cgi/media.cgi?msubmenu=videocodecinfo&action=view&EncodingType=MJPEG (changeable)
Get flip mirror rotation	/stw-cgi/image.cgi?msubmenu=flip&action=view
Get max resolution	/stw-cgi/attributes.cgi/attributes
Set channel	/stw-cgi/media.cgi?msubmenu=videoprofilepolicy&action=set&Channel=3&LiveProfile=3&NetworkProfile=1&RecordProfile=2 (changeable)
Get Motion detection config	/stw-cgi/eventsources.cgi?msubmenu=videoanalysis&action=MotionDetection

Description	SUNAPI Request
Set Motion detection config	/stw-cgi/eventsources.cgi?msubmenu=videoanalysis&action=set&DetectionType=MotionDetection&Sensitivity=High (changeable)
Get video analysis config	/stw-cgi/eventsources.cgi?msubmenu=videoanalysis&action=view&Channel=0
Set video analysis config	/stw-cgi/eventsources.cgi?msubmenu=videoanalysis&action=set&DetectionType=IntelligentVideo&MinimumObjectSize=0,0&MaximumObjectSize=20,20 (changeable)
Get Audiodetection config	/stw-cgi/eventsources.cgi?msubmenu=audiodetection&action=view
Set Audiodetection config	/stw-cgi/eventsources.cgi?msubmenu=audiodetection&action=set (changeable)
Get tampering detection config	/stw-cgi/eventsources.cgi?msubmenu=tamperingdetection&action=view
Set tampering detection config	/stw-cgi/eventsources.cgi?msubmenu=tamperingdetection&action=set&Enable=True&Sensitivity=High (changeable)
Get mount size	/stw-cgi/system.cgi?msubmenu=sdcardinfo&action=view
Get storage info	/stw-cgi/system.cgi?msubmenu=storageinfo&action=view
Set storage info	/stw-cgi/system.cgi?msubmenu=storageinfo&action=set&Enable=True&Storage=1&IsDASEncryptEnable=True (changeable)
Get record config	/stw-cgi/recording.cgi?msubmenu=storage&action=view
Set record config	/stw-cgi/system.cgi?msubmenu=storageinfo&action=set&Enable=True&Storage=1&IsDASEncryptEnable=True (changeable)
Search recorded video	-
Add storage format	/stw-cgi/system.cgi?msubmenu=storageinfo&action=control&Storage=1&Mode=Format (changeable)
Get NAS config	/stw-cgi/system.cgi?msubmenu=storageinfo&action=view

Description	SUNAPI Request
Set NAS config	/stw-cgi/system.cgi?msubmenu=storageinfo&action=set (changeable)
Remove NAS config	/stw-cgi/system.cgi?msubmenu=storageinfo&action=control&Storage=1&Mode=Format (changeable)
Start NAS Record	/stw-cgi/system.cgi?msubmenu=storageinfo&action=control&Storage=1&Mode=NASStart (changeable)
Start using SDCard	/stw-cgi/recording.cgi?msubmenu=storage&action=set&Enable=True
Stop using SDCard	/stw-cgi/recording.cgi?msubmenu=storage&action=set&Enable=False
Format SDCard	/stw-cgi/system.cgi?msubmenu=storageinfo&action=control&Storage=1&Mode=Format (changeable)
Get SDCard status	/stw-cgi/system.cgi?msubmenu=storageinfo&action=view
Get device date time	/stw-cgi/system.cgi?msubmenu=date&action=view
Set Timezone	/stw-cgi/system.cgi?msubmenu=date&action=set&TimeZoneIndex=02 (changeable)
Get system config	/stw-cgi/system.cgi?msubmenu=deviceinfo&action=view
Get camera model	/stw-cgi/system.cgi?msubmenu=deviceinfo&action=view
Get MAC address	/stw-cgi/system.cgi?msubmenu=deviceinfo&action=view
Get privacy mask info	/stw-cgi/image.cgi?msubmenu=privacy&action=view&Channel=0 (changeable)
Set privacy mask info	/stw-cgi/image.cgi?msubmenu=privacy&action=set&Enable=True (changeable)
Add privacy mask area	/stw-cgi/image.cgi?msubmenu=privacy&action=add&MaskColor=White&MaskCoordinate=70,140,817,807,1000,697,1140,420&MaskName=stwcamera (changeable)
Remove privacy mask area	/stw-cgi/image.cgi?msubmenu=privacy&action=remove&MaskIndex=1 (changeable)

Description	SUNAPI Request
Read manifest file	/stw-cgi/opensdk.cgi?msubmenu=manifest&action=view&AppID=SampleApp&Channel=0 (changeable)
Get open platform version	/stw-cgi/system.cgi?msubmenu=deviceinfo&action=view
Get app list	/stw-cgi/opensdk.cgi?msubmenu=apps&action=view

- Example: Requests /stw-cgi/opensdk.cgi?msubmenu=apps&action=view. (See the example of "SUNAPI Response".)

```cpp
std::string sunapi_request = "/stw-cgi/opensdk.cgi?msubmenu=apps&action=view";
SendReplyEvent("SunapiRequester",
               (int32_t)ISunapiRequester::EEventType::eRequestSunapi,
               0,
               new ("")String(sunapi_request),
               (int32_t)ISunapiRequester::EEventType::eRequestSunapiDone
               );
```

4.2.2. Receivable Events

4.2.2.1. SUNAPI Response

- Parameters
  - String

Variable	Type	Description
response	string(Json or xml)	Response to the SUNAPI request

- Example: /stw-cgi/opensdk.cgi?msubmenu=apps&action=view Receiving results for a request. (See the example of "Request SUNAPI".)

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case (int32_t)ISunapiRequester::EEventType::eRequestSunapiDone:
    {
      if(event->IsReply())
      {
        std::string response(reinterpret_cast<String*>(event->GetBaseObjectArgument())->c_str());
        std::cout << response.c_str() << std::endl;
      }
      break;
    }
    ...
  }
  ...
}
```

  - Output result

```json
{"Apps":[{"AppID":"WiseAI","Status":"Running","InstalledDate":"Wed Apr 26 11:11:38
2023","Version":"2.0","AutoStart":true,"Priority":"High","Permission":["None"],"Chnnel":[0]}],"InstalledApps":1}
```

4.2.2.2. Ready for Use SUNAPI

With this event, you can send the **SUNAPI CGI** command. But it is not available as soon as the app starts running. When it becomes available, the SUNAPI Requester sends a called eReadyForUse event.

- Parameters
  None.
- Example

```cpp
bool SampleComponent::ProcessAEvent(Event* event) {
  switch (event->GetType()) {
    case (int32_t)ISunapiRequester::EEventType::eReadyForUse:
    {
      // Sunapi Requester is ready for use. You can send SUNAPI Request.
      ...
      break;
    }
    ...
  }
  ...
}
```

5. Event API Reference Table

5.1. Input Event API Reference Table

Event Descrption	Event ID	Event Relationship	Instance Name	Group Name	Header
Raw Video	IPStreamProviderManagerVideoRaw::EEventType::eVideoRawData	Source-Target	SPMgrVideoRaw_0 ~ SPMgrVideoRaw_#	GroupSPMgrVideoRaw2	i_pl_video_frame_ra
Raw Audio	IPStreamProviderManagerAudioRaw::EEventType::eAudioRawData	Source-Target	SPMgrAudioRaw	SRMgrAudioRawGroup2	i_p_stream_provide
Motion Detection	IAnalyticsDetector::EEventType::eNotifyEventOccured	Source-Taget	MotionDetector_0 ~ MotionDetector_#	OpenSDK::Analytics	i_analytics_detector
Tampering detection	IAnalyticsDetector::EEventType::eNotifyEventOccured	Source-Taget	TamperingDetector_0 ~ amperingDetector_#	OpenSDK::Analytics	i_analytics_detector
Audio detection	IAnalyticsDetector::EEventType::eNotifyEventOccured	Source-Taget	AudioDetector	i_analytics_detector.h	OpenSDK::Analytics
Sound classification	IAnalyticsDetector::EEventType::eNotifyEventOccured	Source-Taget	SoundClassification	i_analytics_detector.h	OpenSDK::Analytics
Defocus detection	IAnalyticsDetector::EEventType::eNotifyEventOccured	Source-Taget	DefocusDetector_0 ~ DefocusDetector_#	OpenSDK::Analytics	i_analytics_detector
Alarm in	IConfigurableAlarmIn::EEvent::eNotifyEventOccured	Source-Taget	ConfigurableAlarmIn	OpenSDK::AlarmIO	i_configurable_alar
Alarm out	IConfigurableAlarmOut::EEvent::eNotifyEventOccured	Source-Taget	ConfigurableAlarmOut	OpenSDK::AlarmIO	i_configurable_alar

Event Descrption	Event ID	Event Relationship	Instance Name	Group Name	Header
ONVIF metadata	IPMetadataManager::EEventType::eMetadataRequest	Source-Taget	MetadataManager_0 ~ MetadataManager_#	Metadata::OpenApp	i_p_metadata_mana
SD Card Inserted	IDeviceIoManager::EEventType::eSdcardInserted	Source-Taget	DeviceIoManager	DeviceIoManager::Sdcard	i_device_io_manage
SD Card Removed	IDeviceIoManager::EEventType::eSdcardRemoved	Source-Taget	DeviceIoManager	DeviceIoManager::Sdcard	i_device_io_manage
SD Card Recording Started	IPOpenPlatformManager::EAppEventType::eSDCardStarted	Source-Taget	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_
SD Card Recording Stopped	IPOpenPlatformManager::EAppEventType::eSDCardStopped	Source-Taget	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_
Network Interface	IPOpenPlatformManager::EAppEventType::eNetworkSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Network Port	IPOpenPlatformManager::EAppEventType::eUpdateAuthInfo	Source-Target	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_
Video Profile	IPOpenPlatformManager::EAppEventType::eProfileSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Audio Config	IPOpenPlatformManager::EAppEventType::eAudioSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Image Config	IPOpenPlatformManager::EAppEventType::eImageSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Video Rotation Config	IPOpenPlatformManager::EAppEventType::eUpdateVideoRotation	Source-Target	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_
Wisestream Config	IPOpenPlatformManager::EAppEventType::eWiseStreamSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Exposure AI Config	IPOpenPlatformManager::EAppEventType::eExposureAISettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Storage Config	IPOpenPlatformManager::EAppEventType::eRecordSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Language Config	IPOpenPlatformManager::EAppEventType::eUpdateSystemSetting	Source-Target	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_

Event Descrption	Event ID	Event Relationship	Instance Name	Group Name	Header
Time Config	IPOpenPlatformManager::EAppEventType::eTimeSettingChanged	Source-Target	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_
Motion Detection Config	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Tampering Detection Config	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Audio Detection Config	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Sound Classification Config	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
Defocus Detection Config	IPOpenPlatformManager::EAppEventType::eAnalyticsSettingChanged	Source-Target	EComponents::eOpenPlatform	SettingChange	i_p_open_platform_
App Warning	IPOpenPlatformManager::EAppEventType::eAppWarning	Source-Target	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_
Stop Application	IPOpenPlatformManager::EAppEventType::eAppStop	Source-Target	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_
Uploaded File Name	IPOpenPlatformManager::EAppEventType::eUploadFile	Source-Target	EComponents::eOpenPlatform	AppMessage	i_p_open_platform_

5.2. Output Event API Reference Table

Event Descrption	Event ID	Event Relationship	Instance Name	Header
Alarm Output On	IConfigurableAlarmOut::EEvent::eRelayRequest	Sender-Receiver	ConfigurableAlarmOut	i_configurable_alarm_out.h
Alarm Output Off	IConfigurableAlarmOut::EEvent::eRelayRequest	Sender-Receiver	ConfigurableAlarmOut	i_configurable_alarm_out.h
FTP File Upload	IPOpenPlatformManager::EAppEventType::eAppFtpFileUpload	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h

Event Descrption	Event ID	Event Relationship	Instance Name	Header
Send Email Notification	IPOpenPlatformManager::EAppEventType::eAppSendEmailNoti	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h
JPEG Encode	IPOpenPlatformManager::EAppEventType::eAppJpegEncode	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h
Set OSD Message	IPStreamRequesterManagerVideo::EEventType::eExternalMultiLineOsd	Sender-Receiver	SRMgrVideo_0 ~ SRMgrVideo_#	i_p_stream_requester_manager_vide
Send Event Metadata	IMetadataManager::EEventType::eRequestMetadata	Sender-Receiver	MetadataManager_0 ~ MetadataManager_#	i_metadata_manager.h
Send Frame Metadata	IMetadataManager::EEventType::eRequestMetadata	Sender-Receiver	MetadataManager_0 ~ MetadataManager_#	i_metadata_manager.h
Write Event Log	ILogManager::EEvent::eRemoteDebugMessage	Sender-Receiver	2009004	i_log_manager.h
Add Dynamic Event Schema(Metadata Schema)	I_EventstatusCGIDispatcher::EEventType::eMetadataSchema	Sender-Receiver	EventstatusCGIDispatcher	i_eventstatus_cgi_dispatcher.h
Add Dynamic Event Schema(Eventstatus Schema)	I_EventstatusCGIDispatcher::EEventType::eEventstatusSchema	Sender-Receiver	EventstatusCGIDispatcher	i_eventstatus_cgi_dispatcher.h
Send Dynamic Event	IMetadataManager::EEventType::eRequestRawMetadata	Sender-Receiver	MetadataManager_0 ~ MetadataManager_#	i_metadata_manager.h
Set Metaframe Schema	I_OpenSDKCGIDispatcher::EEventType::eMetaFrameSchema	Sender-Receiver	Stub::Dispatcher::OpenSDK	i_opensdk_cgi_dispatcher.h
Set Metaframe Options	I_OpenSDKCGIDispatcher::EEventType::eMetaFrameCapability	Sender-Receiver	Stub::Dispatcher::OpenSDK	i_opensdk_cgi_dispatcher.h
Start SD Card	IPOpenPlatformManager::EAppEventType::eStartSDCard	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h
Stop SD Card	IPOpenPlatformManager::EAppEventType::eStopSDCard	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h
Format SD Card	IPOpenPlatformManager::EAppEventType::eFormatSDCard	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h
Set Exclusive Mode	IPOpenPlatformManager::EAppEventType::eExclusiveMode	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h

Event Descrption	Event ID	Event Relationship	Instance Name	Header
Get Exclusive Mode Status	IPOpenPlatformManager::EAppEventType::eGetExclusiveModeStatus	Sender-Receiver	EComponents::eOpenPlatform	i_p_open_platform_manager.h
Rule Instance Registration	IRuleManager::EEventType::eRegisterInstance	Sender-Receiver	IRuleManager::RuleInstanceRequest	i_rule_manager.h
Rule Instance Update	IRuleManager::EEventType::eUpdateInstance	Sender-Receiver	IRuleManager::RuleInstanceRequest	i_rule_manager.h
Rule Instance Unregistration	IRuleManager::EEventType::eUnregisterInstance	Sender-Receiver	IRuleManager::RuleInstanceRequest	i_rule_manager.h

5.3. App Support Package API Reference Table

Event Descrption	Event ID	Event Relationship	Instance Name	Group Name	Header
New Client Connected	IAppNetworkManager::EEventType::eNewClientConnected	Source-Target	AppNetworkManager	NetworkEvent	i_app_network_manager.h
Client Disconnected	IAppNetworkManager::EEventType::eClientDisconnected	Source-Target	AppNetworkManager	NetworkEvent	i_app_network_manager.h
Client Data	IAppNetworkManager::EEventType::eClientData	Source-Target	AppNetworkManager	NetworkEvent	i_app_network_manager.h
Server Data	IAppNetworkManager::EEventType::eServerData	Source-Target	AppNetworkManager	NetworkEvent	i_app_network_manager.h
Start Service	IAppNetworkManager::EEventType::eStartService	Sender-Receiver	AppNetworkManager	X	i_app_network_manager.h
Stop Service	IAppNetworkManager::EEventType::eStopService	Sender-Receiver	AppNetworkManager	X	i_app_network_manager.h
Send Data	IAppNetworkManager::EEventType::eSendData	Sender-Receiver	AppNetworkManager	X	i_app_network_manager.h
Close Client	IAppNetworkManager::EEventType::eCloseClient	Sender-Receiver	AppNetworkManager	X	i_app_network_manager.h
Ready for Use SUNAPI	ISunapiRequester::EEventType::eReadyForUse	Source-Target	SunapiRequester	ReadyForUse	i_sunapi_requetser.h
Request SUNAPI	ISunapiRequester::EEventType::eRequestSunapi,eRequestSunapiDone	Sender-Receiver	SunapiRequester	X	i_sunapi_requetser.h